

In Chap. 5, we dealt with regression models, the first group of so-called supervised models, and applied them to datasets with continuous, numeric target variables, the common data for these kinds of models. We then dedicated Chap. 7 to unsupervised data and described various cluster algorithms. In this chapter, we return to the supervised methods and attend to the third big group of data mining methods, the classification algorithms.

Here, we are confronted with the problem of assigning a category to each input variable vector. As the name suggests, the target variable is categorical, e.g., “Patient is ill” or “Patient is healthy”. As in this example, the target variable’s possible values of a classification problem can’t often be ordered. These kinds of problems are very common in all kinds of areas and fields, such as Biology, Social Science, Economics, Medicine, and Computer Science; almost any problem you can think of that involves deciding between different kinds of discrete outcome.

Figure 8.1 shows the outline of the chapter structure. In the first section, we describe some real-life classification problems in more detail, where data mining and classification methods are useful. After these motivating examples, we explain in brief the concept of classification in data mining, using the basic mathematical theory that all classification algorithms have in common. Then in the remaining sections, we focus on the most famous classification methods, which are provided by the SPSS Modeler, individually, and describe their usage on data examples. A detailed list of the classification algorithms discussed here is displayed in the subsequent Fig. 8.6. Besides the algorithms presented in this book, further classification models are included in the SPSS Modeler. However, a description of these models and their corresponding nodes in the Modeler is omitted, as this would exceed the scope of this book. See Sect. 8.2.2 for more details on these left-out methods and references to the literature.

After finishing this chapter, the reader . . .

1. is familiar with the most common challenges when dealing with a classification problem and knows how to handle them.

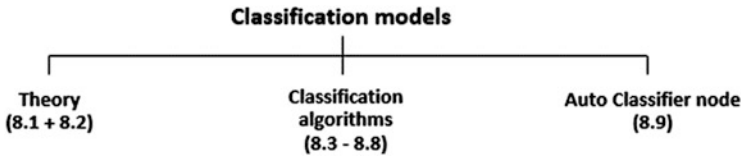


Fig. 8.1 Outline of the chapter structure

2. possesses a large toolbox of different classification methods and knows their advantages and disadvantages.
3. is able to build various classification models with the SPSS Modeler and is able to apply them to new data for prediction.
4. knows various validation methods and criteria and can evaluate the quality of the trained classification models within the SPSS Modeler stream.

8.1 Motivating Examples

Classification methods are needed in a variety of real world applications and fields, and in many cases, they are already utilized. In this chapter, we present some of these applications as motivating examples.

Example 1

Diagnosis of Breast Cancer

To diagnose breast cancer, breast mass is extracted from numerous patients by fine needle aspiration. Each sample is then digitalized into an image from which different features can be extracted. Among others, some include the size of the cell core or the number of mutated cells. The feature records of the patients, together with the target variable (cancer or no cancer), are then used to build a classification model. In other words, the model learns the configuration and structure of the features for tumorous and non-tumorous tissue samples. Now, for a new patient, doctors can easily decide whether or not breast cancer is present, by establishing the above-mentioned features and putting these into the trained classifier.

This is a classical application of a classification algorithm, and logistic regression is a standard method used for this problem. See Sect. 8.3 for a description of logistic regression.

An example dataset is the Wisconsin Breast Cancer Data (see Sect. 12.1.40). Details on the data, as well as the research study, can be found in Wolberg and Mangasarian (1990).

Example 2

Credit Scoring of Bank Customers

When applying for a loan at a bank, your credit worthiness is calculated based on some personal and financial characteristics, such as age, family status, income, number of credit cards, or amount of existing debt. These variables are used to estimate a personal credit score, which indicates the risk to the bank when giving you a loan.

An example of credit scoring data is the “tree_credit” dataset. For details, see Sect. 12.1.38.

Example 3

Mathematical Component of a Sleep Detector (Prediction of Sleepiness in EEG Signals)

Drowsiness brings with it a lesser ability to concentrate, which can be dangerous and should be avoided in some everyday situations. For example, when driving a car, severe drowsiness is the precursor of microsleep, which can be life-threatening. Moreover, we can think of jobs where high concentration is essential, and a lack of concentration can lead to catastrophes. For example, the work of plane pilots, surgeons, or technique observers at nuclear reactors. This is one reason why scientists are interested in detecting different states in the brain, to understand its functionality.

For the purpose of sleep detection, EEG signals are recorded in different sleep states, drowsiness, and full consciousness, and these signals are analyzed to identify patterns that indicate when drowsiness may occur. The EEG_Sleep_Signals.csv (see Sect. 12.1.12) is a good example, which is analyzed with a SVM in Sect. 8.5.2.

Example 4

Handwritten Digits and Letter Recognition

When we send a letter through the mail, this letter is gathered together with a pile of other letters. The challenge for the post office is to sort this huge mass of letters by their destination (country, city, zip-code, street). In former days, this was done by hand, but nowadays computers do this automatically. These machines scan the address on each letter and allocate it to the right destination. The main challenge is that handwriting differs noticeably between different people. Today’s sorting machines use an algorithm that is able to recognize alphabetical characters and numbers from individual people, even if it has never seen the handwriting before. This is a fine example of a machine learning model, trained on a small subset and able to generalize to the entire collective.

Other examples where automatic letter and digit identification can be relevant are signature verification systems or bank-check processing. The problem of handwritten character recognition falls into the more general area of pattern recognition. This is a huge research area with many applications, e.g., automatic identification of the correct image of a product in an online shop or analysis of satellite images.

The optical recognition of handwritten digits data obtained from the UCI Machine Learning Repository Machine Learning Repository (1998) is a good example. See also Sect. 12.1.29.

Other Examples and Areas Where Classification Is Used

- Sports betting. Prediction of the outcome of a sports match, based on the results of the past.
- Determining churn probabilities. For example, a telecommunication company wants to know if a customer has a high risk of switching to a rival company. In this case, they could try to avoid this with an individual offer to keep the customer.

- In the marketing area: to decide if a customer in your database has a high potential of responding to an e-mailing marketing campaign. Only customers with a high response probability are contacted.

8.2 General Theory of Classification Models

As described in the introduction to this chapter, classification models are dedicated to categorizing samples into exactly one category. More precisely, let y be the observation or target variable that can take one of a finite number of values $A, B, C, \dots$. Recalling the motivating examples in the previous section, these values can, for example, describe a medical diagnosis, check if an e-mail is spam, or discern the possible meaning of a handwritten letter. As these examples show, the observation values do not have to be in any kind of order or even numeric.

Based on some input variables, $x_{i1}, \dots, x_{ip}$, of a sample i , a classification model now tries to determine the value of the observation y_i , and thus, categorize the data record. For example, if some breast sample tissue has unusual cell sizes, this could likely be cancer. Or, if the subject of an e-mail contains some words that are common in spam, this e-mail is probably a spam mail too.

In this chapter, we give a brief introduction on the general theory of classification modeling.

8.2.1 Process of Training and Using a Classification Model

The procedure for building a classification model follows the same concept as regression modeling (see Chap. 5). As described in the regression chapter, the original dataset is split into two independent subsets, the *training* and the *test set*. A typical partition is 70% training data and 30% test data. The training data is used to build the classifier, which is then applied to the test set for performance evaluation on unknown data. Using this separate test dataset is the most common way to verify the universality of the classifier and its prediction ability in practice. This process is called *cross-validation* (see Sect. 5.1.2 or James et al. (2013)). We thus recommend always using this training and test set framework when building a classifier.

Often, some model parameters have to be predefined. These, however, are not always naturally given and have to be chosen by the data miner. To find the optimal parameter, a third independent dataset is used, the *validation set*. A typical partition of the original data is then 60% training, 20% validation, and 20% test set. After training several classifiers with different parameters, the validation set is used to evaluate these models in order to find the one with the best fit (see Sect. 8.2.5 for evaluation measures). Afterwards, the winner of the previous validation is applied to the test set, to measure its predicting performance on new data. This last step is necessary to eliminate biases that might have occurred in the training and validation steps. For example, it is possible that a particular model performs very well on the

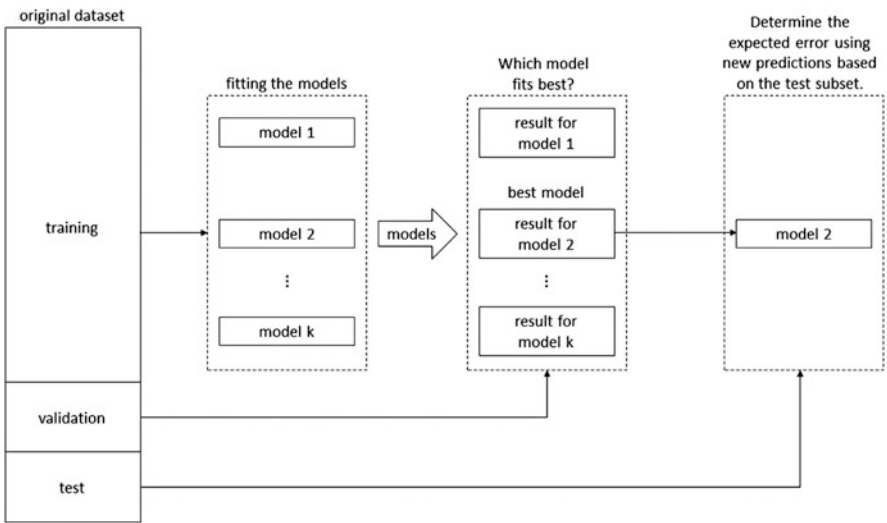


Fig. 8.2 Overview of the steps when building a classification model

validation set, but is not very good on other datasets. In Fig. 8.2, the process of cross-validation and building a prediction model is illustrated.

We further like to mention that, in the area of big data, where datasets contain millions of samples, the size of the test and validation set can be chosen much smaller. In this situation, a size of 10% or lower for the validation and test set is not unusual, as these partitions still contain enough data to evaluate the model. Even a size of 1% is possible if sufficient data are available. However, the test and validation sets have to be chosen large enough so that the model is properly validated.

The general idea behind a classification model is pretty simple. When training a classification model, the classification algorithm inspects the training data and tries to find a common structure in data records with the same target value and differences between data records of different target values. In the simplest case, the algorithm converts these findings into a set of rules, such that the target classes are characterized in the best possible way through these “if . . . then . . .” statements.

In Fig. 8.3, this is demonstrated with a simple data example. There, we want to predict if we can play tennis this afternoon based on some weather data. The classification algorithm transforms them into a set of rules that define the classifier. For example, if the outlook is “Sunny” and the temperature is greater than 15 °C, we will play tennis after work. If otherwise the outlook is “Rain” and the wind prognosis is “strong”, we are not going to play tennis.

Applying a classifier to new and unseen data now simply becomes the assignment of a class (target value) to each data record using the set of rules. For example, in Fig. 8.4, the classifier that suggests whether we can play tennis is applied to new data. For day 10, where the Outlook = “Sunny”, Temperature = 18 °C, and

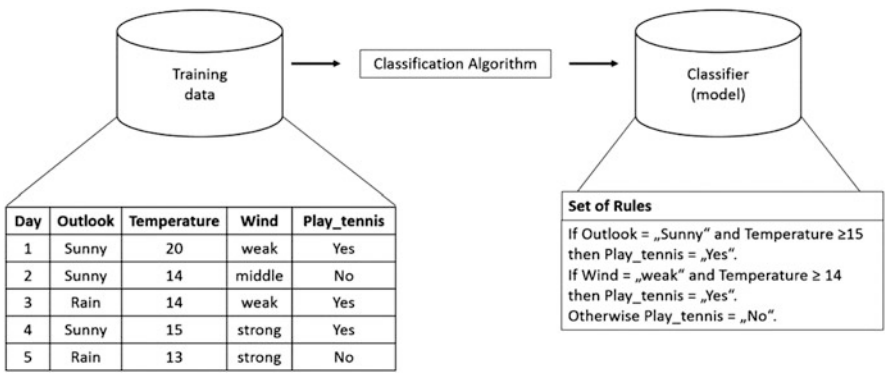


Fig. 8.3 Training of a classification model

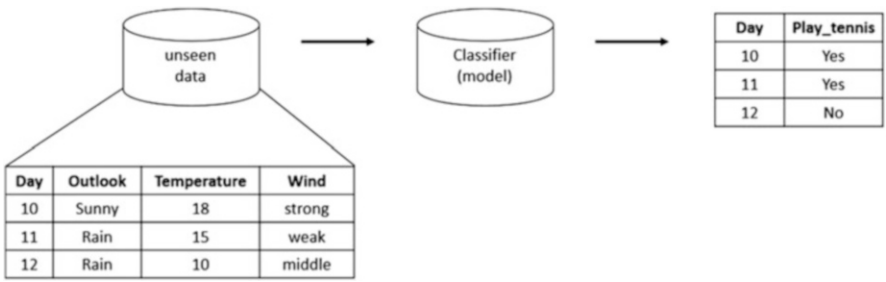


Fig. 8.4 Prediction with a classification model

Wind = “strong”, the classifier predicts that we can play tennis in the afternoon, since these variable values fulfill the first of its rules (see Fig. 8.3).

Many classification models preprocess the input data via a numeric function, which is used to transform and quantify each data record. Each data record is then assigned a so-called score, which can be interpreted as the probabilities of each class, or some distances. When training the model, this scoring function and its parameters are determined, and a set of decision rules for the function’s value is generated. The class of unseen data is now predicted by calculating the score with the scoring function and assigning the target class suggested by the score. See Fig. 8.5 for an illustration of a classification model with an internal scoring function.

8.2.2 Classification Algorithms

A variety of classification algorithms and concepts exist, with different approaches and techniques to handle the data and find classification rules. These methods can be roughly grouped into three types, *linear*, *nonlinear*, and *rule-based* algorithms. The models of the first two types follow a mainly mathematical approach and generate

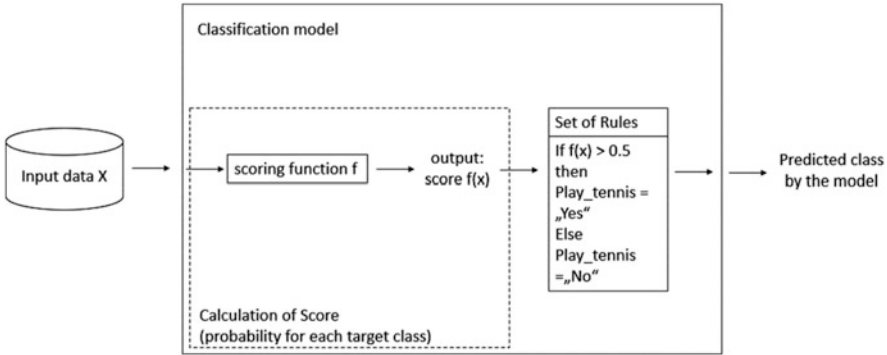


Fig. 8.5 Classification model with scoring function

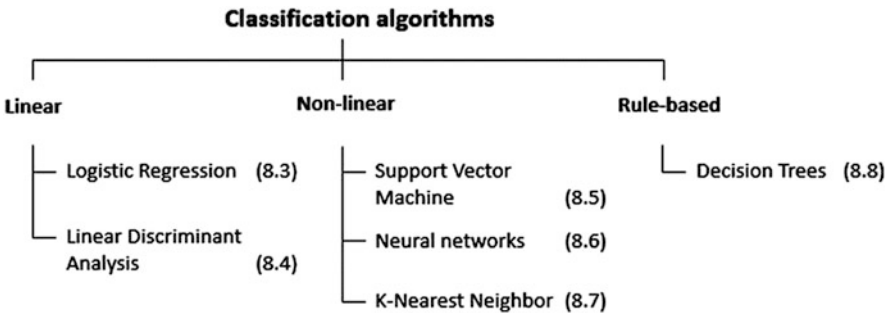


Fig. 8.6 Overview of the classification algorithms presented in this book

functions for scoring and separation of the data and classes. While linear methods try to separate the different classes with linear functions, nonlinear classifiers can construct more complex scoring and separating functions. In contrast to these mathematical approaches, the rule-based models search the input data for structures and commonalities without transforming the data. These models generate “if . . . then . . .” clauses on the raw data itself. Figure 8.6 lists the most important and favored classification algorithms that are also implemented in the SPSS Modeler and discussed in this book. Chapters where the particular models are discussed are shown in brackets.

In addition to the models presented in this book, the SPSS Modeler provides further classification models and nodes, e.g., Random Tree and Bayesian Network. However, a description of these models would exceed the scope of this book and is therefore omitted, and we refer the interested reader to Tuffery (2011), Han (2012), and IBM (2019b) resp. IBM (2019a) for information on the nodes in the SPSS Modeler and algorithm details. Among these algorithms, the random forest is the most important one with very powerful prediction abilities. It is highly popular in the data mining community, and thus often used for classification problems. A

description of the method and how a random forest model can be trained with the SPSS Modeler is provided through the corresponding website of this book “<http://www.statistical-analytics.net>”. See Chap. 1 for details to access the content on the website related to this book.

The SPSS Modeler further comprises nodes for building boosting models, e.g., XGBoost Tree. Ensemble methods, and in particular boosting, are a common concept in classification (see Sect. 8.8.1). Since many of the regular nodes, which are discussed in this book, itself provide options to build ensemble and boosting models, we skip a description of the nodes explicitly dedicated to boosting and refer to IBM (2019b) for details on these nodes, and Zhou (2012) for the boosting concept.

Moreover, some classification algorithms are implemented in the SPSS Modeler 18.2 to run on the SPSS Analytics Server (see IBM (2019d)) and provided by the Modeler in separate nodes, like the Tree-AS node. Since these nodes are more or less an adaptation of the nodes described in this chapter, we will ignore them here and point to IBM (2019b) for details.

Each classification model has its advantages and disadvantages. There is no method that outperforms every other model for every kind of data and classification problem. The right choice of classifier strongly depends on the data type and size. For example, some classifiers are more applicable to small datasets, while others are more accurate on large data. Other considerations when choosing a classifier are the properties of the model, such as accuracy, robustness, speed, and interpretability of the results. A very accurate model is never bad, but sometimes a robust model, which can be easily updated and is insensitive to strange and unusual data, is more important than accuracy. In Table 8.1, some advantages and disadvantages of the classification algorithms are listed to give the reader guidelines for the selection of an appropriate method. See also Tuffery (2011). However, we strongly recommend to try different approaches and methods on each dataset in order to see what will work best for the particular problem and data. The SPSS Modeler provides an easy way for this task of building and comparing different models on one dataset, the Auto Classifier node, which will be explained in detail in Sect. 8.9.2.

8.2.3 Classification Versus Clustering

Recalling the discussion in Sect. 7.2, a clustering algorithm is an *unsupervised* learning method. This means that the data points are not labeled, and hence, there is no target variable present. Clustering is therefore used to group the data points, based on their similarities and dissimilarities. The purpose is to find new patterns and structures within the data. Recall the motivating example of Customer segmentation in Sect. 7.1; customer segmentation is often used by companies to identify high potential and low potential customers. For example, banks cluster their customers into groups to identify the customers with high or low risk of loan default.

As described in the previous sections, classification has a different purpose. When using a classifier, the data points are labeled. In the previous example of loan default risk, the bank already has a dataset of customers, each of them labeled “risky” or

Table 8.1 Overview of advantages and disadvantages of the classification algorithms

	+	—
Logistic regression	- Models are often very accurate - Works well on small datasets - Predicts probabilities - Easy to interpret, in particular, the influence of each input variable	- It can only provide linear solutions - Problems with high collinearity of the input variables
Linear discriminant analysis	- Typically, very fast building the model - Works well on small datasets - Optimal if data assumptions are fulfilled	- More restrictive assumptions than other methods (e.g., logistic regression) - Usually, needs data preparation - Sensitive to outliers - Only applicable to linear problems
Support vector machine	- It can easily handle complex (nonlinear) data and a large feature space - Typically a high accuracy - Less susceptible to overfitting than other methods	- Long training time for large datasets - Model is hard to interpret (black box algorithm) - Sensitive with regard to the choice of kernel function and parameter - Choosing the wrong kernel and parameters can risk overfitting - Sensitive to outliers
Neural network	- Good performance on large datasets - Very good at allowing nonlinear relations and can generate very complex decision boundaries - Nonparametric. No distribution assumptions needed - Can handle noisy data - Often outperforms other methods	- Training can be computationally expensive - Results and effects of input variables are hard to interpret (black box algorithm) - Tends to overfitting - Does not always find the optimal solution - Can only process continuous input variables
K-Nearest neighbor	- Very fast training - Complex concepts can be learned through simple procedures - Intuitive algorithm - Only few parameters to tune	- Requires large storage space - Performance depends on the number of dimensions (curse of dimensionality) - Problem with highly imbalanced data - No interpretation of results possible - Slow in prediction for large datasets - Sensitive to noisy data. Typically, needs scaling and feature selection preprocessing
Decision trees	- Robust to outliers - Model and decision rules are easy to understand - Can handle different data types, i.e., numerical and categorical variables	- Can be computationally expensive to train - Large trees tend to overfitting - Most of the time it does not find the optimal solution

(continued)

Table 8.1 (continued)

	+	–
	• Fast in prediction • No assumptions on variable distributions needed. Thus, less effort for data preparation. • Can handle missing values	• Prefers variables with many categories or numerical data
Random Forest	• It has the same advantages as a decision tree since it comprises multiple trees • Typically results in a better model than one decision tree • Good performance on small datasets • Works great with high dimensionality (many input variables) • Fast training and prediction	• Overfitting is more likely than with other methods • The model is hard to interpret (black box algorithm) • Can take up a lot of memory for large datasets

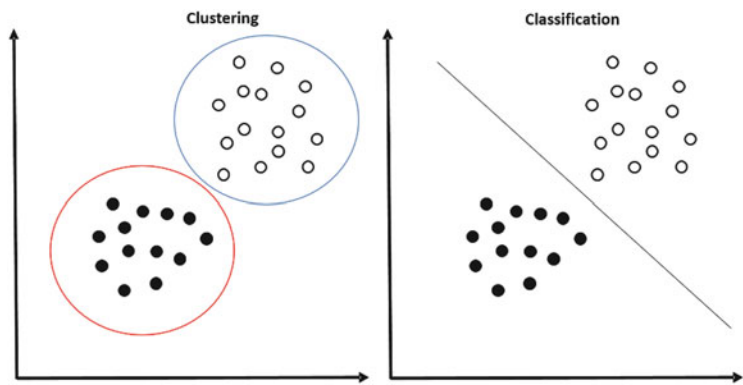


Fig. 8.7 Clustering vs. classification

“non-risky”, based on the customers’ ability to repay his/her loan. Based on this training data of bank customers, a classification model learns to separate the “risky” from the “non-risky” customers. Afterwards, new customers can be categorized into risky or non-risky, so the bank has a better basis on which to decide to give a customer a loan. Due to this labeled training data, classification models belong to the *supervised* learning algorithms. The model learns from some labeled training data and then predicts the outcome of the target variable based on this training data.

In Fig. 8.7, the difference between classification and clustering is visualized.

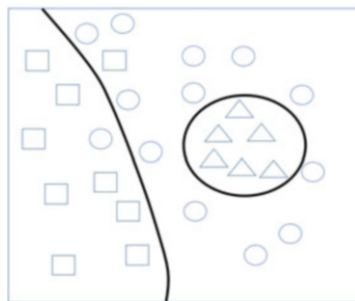
8.2.4 Decision Boundary and the Problem with Over- and Underfitting

The task (mathematically speaking) of a classification model is basically the following: Find a function describing a line (or curve) that separates the different classes from each other in the best possible way. This separating function is called *decision boundary*. The classifier now simply assigns a new data point to a class by looking at which side of the decision boundary it is located. For example, in the right graph in Fig. 8.7 the black circles are separated from the white circles by a straight line. A new data point is now classified as a black circle if it lies underneath the line, and otherwise, it is labeled as a white circle. This is an easy example of a linear classifier, meaning the decision boundary is a straight line, in a binary classification problem. However, the decision boundary can be nonlinear and highly complex, separating multiple classes from each other (see Fig. 8.8).

With the discussion of the decision boundary in mind, a perfect classifier would be the one separating the classes and making no misclassification, i.e., all data points lie on the correct side of the decision boundary. Unfortunately, a perfect classifier with no misclassification is nearly impossible. Now, one can justifiably interject that nearly every dataset of different labeled points can be separated perfectly using a highly complex function. This is undoubtedly true. However, a complex decision boundary might perfectly classify the existing data, but if a new and unknown data point has to be classified, it can easily lie on the wrong side of the decision boundary, and thus, be misclassified. The perfect classifier no longer exists. The problem here is that, instead of describing the underlying structure of the data, the classifier models the noise and thus suffers from *overfitting*. The model is inappropriate to apply to new data, even though it perfectly classified the training data. Thus, we have to reduce our expectations and condone little misclassifications in favor of a simple decision boundary and more universal model, see Fig. 8.8 for an example. See Kuhn and Johnson (2013) for more information on the decision boundary and overfitting, as well as Sect. 5.1.2.

These considerations of overfitting are illustrated in Fig. 8.9. There, the decision boundary in the graph on the left separates the squares from the circles perfectly. In the middle graph, a new data point is added, the filled square, but it lies on the circle side of the boundary. Thence, the decision boundary is no longer optimal. The model

Fig. 8.8 Decision boundary of a three class classifier



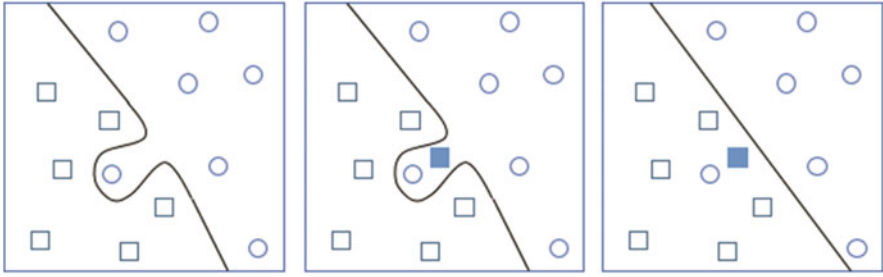


Fig. 8.9 Illustration of the problem of overfitting, when choosing a decision boundary

has not identified the circle in the middle as noise, and thus suffers from overfitting. So, the linear boundary in the graph on the right is as good as the boundary in the middle graph, but describes the underlying structure of the data in a more suitable and, moreover, simpler way.

The common way to detect overfitting is cross-validation. If the model performs much better on the training set than on the test set, overfitting might be the problem (see Sect. 8.2.5 for a list of evaluation measures). So, in order to avoid overfitting, we recommend always using a training/testing setting and potentially using a separate validation set too. If overfitting occurs, there are some techniques that can be utilized to approach this problem:

- *Reduce the number of input variables.* This leads to a simpler decision boundary, and thus, a more general model.
- *Reduce the number of iterations during training.* If the algorithm is using an iterative approach during training, it improves the fit of the model to the training data in each iteration step. Stopping this process in an early iteration run can reduce the complexity of the resulting model and thus prevent overfitting.
- *Use ensemble methods.* Ensemble methods combine multiple separate models and approaches. Hence, the prediction and the fit do not depend on only one model. The most famous ensemble techniques are Boosting and Bagging (see Sect. 5.3.6).

The opposite of a too complex model, which can result in overfitting, is a too simple model that does not capture the underlying data structure sufficiently. This is called *underfitting*, which is illustrated in Fig. 8.10. There, in the left graph, the squares are separated from the circles by a straight line. However, the squares are more concentrated in a cloudy formation and a circle-shaped decision boundary is more appropriate here, as it better describes the underlying structure of the data (see right graph).

An indicator for models that suffer from underfitting is a bad performance on the training set. In this case, the model is probably too simple, and a switch to a more complex algorithm, e.g., nonlinear instead of a linear approach, can improve the prediction performance and overcome underfitting.

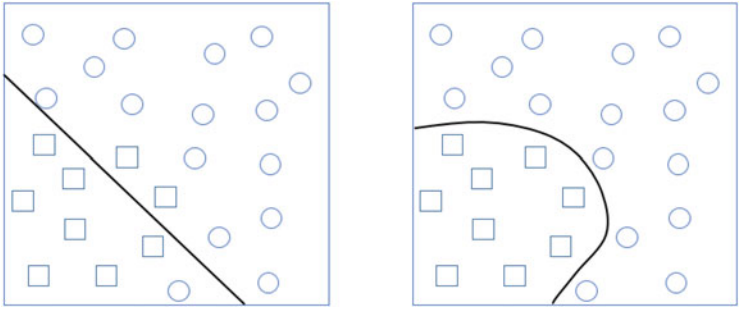


Fig. 8.10 Illustration of the problem of underfitting

Table 8.2 A confusion matrix gives a good overview of the possible classification results

		Predicted category	
		Yes	No
Actual category	Yes	True positive (TP)	False negative (FN)
	No	False positive (FP)	True negative (TN)

8.2.5 Performance Measures of Classification Models

Various measures and methods exist for the evaluation of a classification model. Here, we discuss the most common ones implemented in the SPSS Modeler.

Confusion Matrix

In the case of a binary target variable, i.e., a “yes” or “no” decision, four possible events can occur when assigning a category to the target variable via classification.

- *True positive (TP)*. The true value is “yes” and the classifier predicts “yes”. A patient has cancer and cancer is diagnosed.
- *True negative (TN)*. The true value is “no” and the classifier predicts “no”. A patient is healthy and no cancer is diagnosed.
- *False positive (FP)*. The true value is “no” and the classifier predicts “yes”. A patient is healthy, but cancer is diagnosed.
- *False negative (FN)*. The true value is “yes” and the classifier predicts “no”. A patient has cancer, but no cancer is diagnosed.

These four possible classification results are typically structured in a *confusion matrix*, which displays the number of TP, TN, FP, and FN for the training resp. test set (see Table 8.2).

When predicting the target variable, two errors can be made: FP and FN prediction. These errors are also often referred to as *type I* and *type II error*, in relation to classical hypotheses tests and their nomenclature, where these two errors play an

important role for the design of a statistical test. We refer the interested reader to Dekking et al. (2005) for further information.

Accuracy and Metrics Derived from the Confusion Matrix

A model is said to be a good classifier if it predicts the true value of the outcome variable with high accuracy, that is, if the proportion of TPs and TNs is high, and the number of FPs and FNs is very low. So, the obvious and most common measure for evaluating a classifier is the ratio of correct identified data points, the *accuracy*

$$ACC = \frac{TP + TN}{P + N} = \frac{TP + TN}{TP + FN + TN + FP},$$

with $P = TP + FN$, the number of actual positive (“yes” or 1) values, and $N = TN + FP$, the number of actual negative (“no” or 0) values. Its complement is the rate of misclassified instances, called *classification error*, in formula $1 - ACC$, which should be small for a well-fitted model.

Further measures indicating the quality of the model and its ability to classify the individual classes correctly are the *true positive rate (TPR)*, *false negative rate (FNR)*, *true negative rate (TNR)*, and *false positive rate (FPR)*

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}, \quad FNR = \frac{FN}{P} = \frac{FN}{TP + FN}$$

and

$$TNR = \frac{TN}{N} = \frac{TN}{TN + FP}, \quad FPR = \frac{FP}{N} = \frac{FP}{TN + FP}.$$

The TPR, also called *recall*, *hit rate*, or *sensitivity*, is defined as the ratio of positive values (or 1s) that are correctly classified by the model from all actual 1s. In other words, the TPR describes the probability that a positive value data record is classified as such. So, a model with a high TPR, i.e., near 1, has a high chance in predicting a positive value record correctly. On the other hand, if the TPR is low, i.e., near 0, then the model is inadequate in finding the class 1 data, and further parameter tuning or data preparation need to be considered in order to improve the TPR of the model.

The TNR, also called *specificity*, is the same metric as the TPR, but for the negative value class (or 0s). More precisely, it is defined as the ratio of negative values (or 0s) that are correctly classified by the model from all actual 0s, and so, describes the probability that a data record labeled as 0 is categorized as 0 by the model.

We like to mention that a single one of the four measures is unable to describe the model entirely, and thus, multiple rates should be included in the evaluation process. A high TPR, for instance, doesn’t say anything about the capability of the model detecting the negative value, i.e., 0, class. Let us consider the following example: The goal of a model is to classify a tissue sample as tumorous or healthy. But,

because of our fear missing a cancer disease, we declare every sample as tumorous, including the healthy ones. This procedure leads to a TPR of exactly 1 (all actual positive values are correctly classified), but simultaneously, it generates a lot of FPs or false alarms, i.e., all actual negative valued samples are classified as tumorous. This results in a TNR of 0 resp. FPR of 1. The model is unable to detect any healthy patients and separate the two classes properly. This is an example of the “cry wolf” expression, which refers to a shepherd that cries out “Wolf!” just to entertain himself, even though no wolf is in sight. But, on the day a real wolf is approaching, no one believes him, because of his many false alarms in the past.

The above example describes a poor model, as it completely ignores one class (TPR = 1 and FPR = 1), whereas a good classifier, on the other hand, has a high TPR as well as a low FPR. In practice, however, increasing the TPR often goes along with increasing the FPR as well and vice versa. So, one typically has to settle for a trade-off between the hit rate and false alarm. As an example, we refer to Sect. 8.2.8 where the relationship between the TPR and FPR is discussed in more detail.

ROC and AUC

One of the most popular methods and performance measures that takes the TPR and FPR into account simultaneously, and thus, can quantify the trade-off between hit rate and false alarm, is the *Receiver Operating Characteristic curve (ROC curve)*, and derived from that, the *Area Under this Curve (AUC)*; the latter, one of the most important performance measures for classifiers.

The ROC curve visualizes the false positive rate (FPR) against the true positive rate (TPR). More precisely, consider a binary classifier (0 and 1 as the target categories) with a score function that calculates the probability of the data point belonging to class 1. By default, a model classifies a data point to class 1, if the score is higher than 0.5, and to class 0 if the score is lower 0.5. This is the standard prediction which is returned by the model.

However, for each threshold t between 0 and 1, such a decision function can be constructed that assigns a data point to class 1 if and only if the predicted probability is greater or equal than the threshold, i.e.,

$$\begin{aligned} \text{score} \geq t, & \text{ then } \text{class} = 1 \\ \text{score} < t, & \text{ then } \text{class} = 0. \end{aligned}$$

For each of these thresholds t , the target variables are predicted via the above decision formula and the confusion matrix is calculated, from which the $\text{TPR}(t)$ and the $\text{FPR}(t)$, which depend on t , can be derived. Visualizing these values $\text{TPR}(t)$ and $\text{FPR}(t)$, $0 \leq t \leq 1$, in a graph, they form a curve, the above-mentioned ROC curve. See Fig. 8.14 as an example of a typical ROC curve. Of course, this procedure works analog with every other form of score.

To get a better understanding, let us take a quick look at a toy example. For a more detailed explanation of how the ROC curve is created, we refer to Sect. 8.2.8. Consider the following probability predictions (Fig. 8.11):

Prob. for class 1	True class
0.80	1
0.50	1
0.60	0
0.20	0

Fig. 8.11 Example predictions for ROC

Threshold	0	0.2	0.5	0.6	0.8	1				
TPR	1	1	0.5	0.5	0	0				
FPR	1	0.5	0.5	0	0	0				
Confusion Matrix	2	0	2	0	1	1	1	2	0	2
	2	0	2	0	1	1	1	2	0	2

Fig. 8.12 TPR and FPR for different thresholds

Prob. for class 1	Predicted class	True class
0.80	1	1
0.50	1	1
0.60	1	0
0.20	0	0

Fig. 8.13 Example predictions to calculate the ROC curve for the threshold 0.5

To calculate the ROC curve, the FPR and TPR are determined for the thresholds 1, 0.8, 0.6, 0.5, 0.2, 0. We only have to consider the thresholds that correspond to the predicted probabilities since these are the only values where the confusion matrix and its entries change. The results are displayed in Fig. 8.12, including the confusion matrix that contains the TP, FP, FN, and TN values in the different cases.

As an example, let us take a look at $t = 0.5$. There are three data points with a probability for class 1 at least 0.5; hence, they are assumed to belong to class 1. Two of them are actually of class 1, while one is misclassified (the one with probability 0.6) (see Fig. 8.13). The last data point with probability 0.2 is assigned to class 0 since its predicted probability is smaller than the threshold. With these predicted classes, compared with the true classes, we can construct the confusion matrix and easily calculate the TPR and FPR. See the third column in Fig. 8.12.

The optimal classifier has $TPR = 1$ and $FPR = 0$, which means that a good classification model should have an ROC curve that goes from the origin in a relatively straight line to the top left corner, and then to point (1,1) on the top right. Figure 8.14 shows a ROC curve of an optimal classifier (right graph) and a typical ROC curve (left graph). The diagonal symbolizes the random model, and if

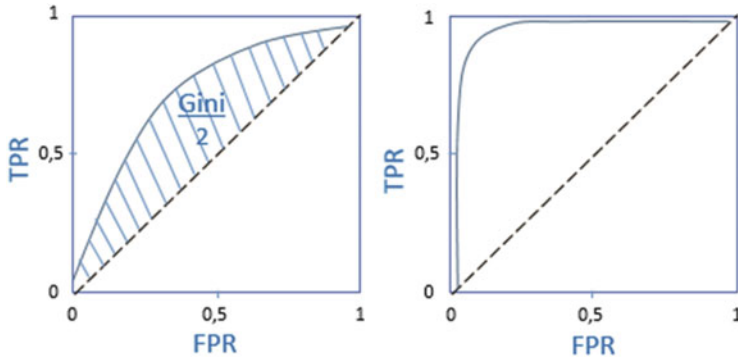


Fig. 8.14 A typical ROC curve is illustrated in the first graph and an optimal ROC curve in the second graph

the curve coincides with it, it doesn't perform any better than guessing. In cases where the ROC curve is under the diagonal, it is most likely that the model classifies the inverse, i.e., 0 is classified as 1 and vice versa.

From the ROC curve, one of the most common and used goodness of fit measures for classification models can be derived: the *Area Under the Curve (AUC)*. It is simply the area under the ROC curve and can take values between 0 and 1. With the AUC, two models can be compared, and a higher value indicates a more precise classifier and thus a more suitable model for the data.

For more detailed information on the ROC curve and AUC, we recommend Kuhn and Johnson (2013) and Tuffery (2011), as well as the explanations in Sect. 8.2.8.

In the SPSS Modeler, the ROC curve can be drawn with the Evaluation node, which is explained in Sect. 8.2.7.

Gini Index

Another important and popular goodness of fit measure for a classification model is the *Gini index*, which is just a transformation of the AUC, i.e.,

$$\text{Gini} = 2 \text{ AUC} - 1.$$

As with the AUC, a higher Gini index indicates a better classification model. The shaded area in the first graph in Fig. 8.14 visualizes the Gini index and shows the relationship with the AUC.

8.2.6 The Analysis Node

The confusion matrix and the performance measures presented in the previous section are implemented in the SPSS Modeler and can be displayed via the Analysis node, along with additional statistics. For this purpose, the Analysis node has to be

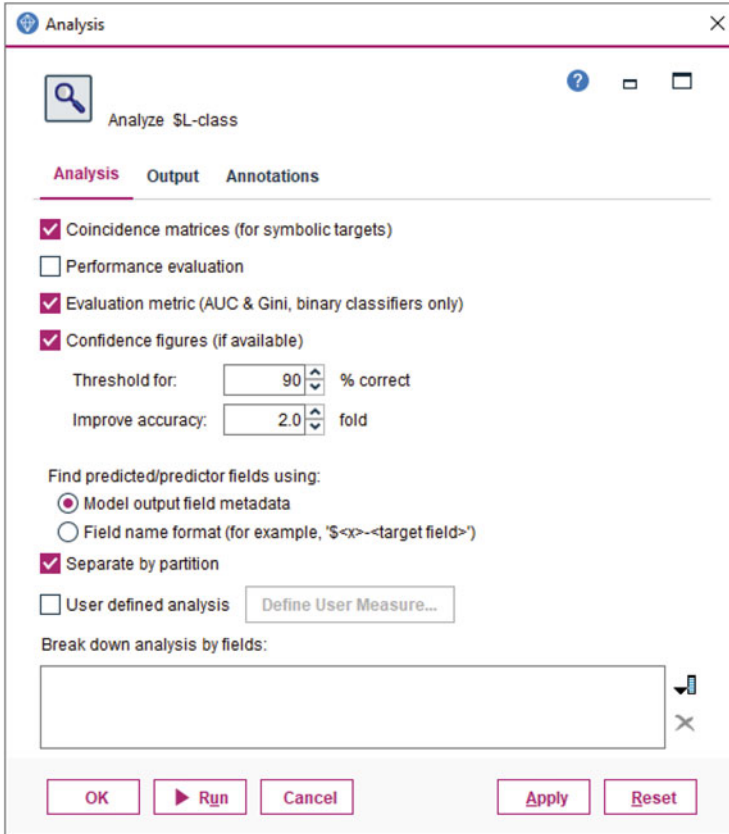


Fig. 8.15 Options in the Analysis node for a classification model

added to the model nugget (the part in the stream that contains the trained model, see, e.g., Chap. 5 or Sect. 8.3.2) and then executed.

In Fig. 8.15 the options of the Analysis node are shown. There, the three most important statistics and measures, that are available for a classification model, are marked: *Coincidence matrices*, *Evolution metric*, and *Confidence figures*. The output of the Analysis node for each of these metrics is described in Table 8.3. In addition to the mentioned statistics, a *Performance Evaluation* can be performed by the Analysis Node, which calculates Kononenko’s and Bratko’s information-based evolution criterion. We will not further address this criterion in this book since it is used rarely in practice. We refer the interested reader to Kononenko and Bratko (1991).

The output of the Analysis node for a binary target variable can be viewed in Fig. 8.16, which shows the statistics for a classifier on the Wisconsin breast cancer data (see stream in Sect. 8.3.6) for the training and the test set separately. This is due to the option “Separate by partition”, which is enabled by default. The accuracy is

Table 8.3 List of the available statistics for a classification model in the Analysis node

Coincidence matrices	Display of the confusion matrix that shows the predicted target values against the true target categories for each data record. This helps to identify systematic prediction errors. See Sect. 8.2.5 for more details on the confusion matrix.
Evaluation metric (AUC & Gini)	Display of the AUC and Gini index. This option can only be chosen for a binary classifier. See Sect. 8.2.5 for more details on the measures. To calculate the values of these measures, the target variable has to be set to flag, as a measurement in the type node. See Fig. 8.17.
Confidence figures	When enabled, some confidence values and statistics describing the distribution of the predicted probabilities are calculated. This option is only available if the model generates a class probability or score as prediction, e.g., logistic regression. For a detailed description of these statistics, see IBM (2019c).

shown at the top of the output, followed by the coincidence resp. confusion matrix and a variety of confidence statistics, which describe the distribution of the predicted scores resp. probabilities of the model. The Gini index and AUC are displayed at the bottom of the Analysis node output. All statistics are individually calculated for all data partitions, i.e., the training and test set, respectively.

The output in the multiple classification case is basically the same, except for the missing AUC and Gini values, which are only implemented for binary target variables. For an example of such a multiple classifier output, see Fig. 8.118.

8.2.7 The Evaluation Node

With the Evolution node, the SPSS Modeler provides an easy way to visualize the ROC curve, along with other evaluation curves. These evaluation charts show the prediction performance of models and are commonly used to compare different models with each other in order to find the best fitted one. To plot such an evaluation chart, one has to connect the Evolution node to the model nugget and afterwards run it.

In Fig. 8.18, the options of the Evaluation node in the Plot tab are shown. In particular, the figure shows the setting for a ROC curve; hence, the Chart type is set to Receiver Operating Characteristic (ROC). The Evaluation node also provides other graphs that can be selected as Chart type: *Gains*, *Response*, *Lift*, *Profit*, *Return on Investment (ROI)*. The description of these graphs is postponed until the end of this section.

In the following, we list the most important options available in the Evaluation node to configure the graphs. We refer to IBM (2019c) for further information.

- *Cumulative plot.* If this option is enabled, the cumulative chart is plotted instead of a graph showing the increments. A cumulative chart is usually smoother, as it neglects the increment fluctuation. For this reason, this option is enabled by default. This option is not available for ROC curves.

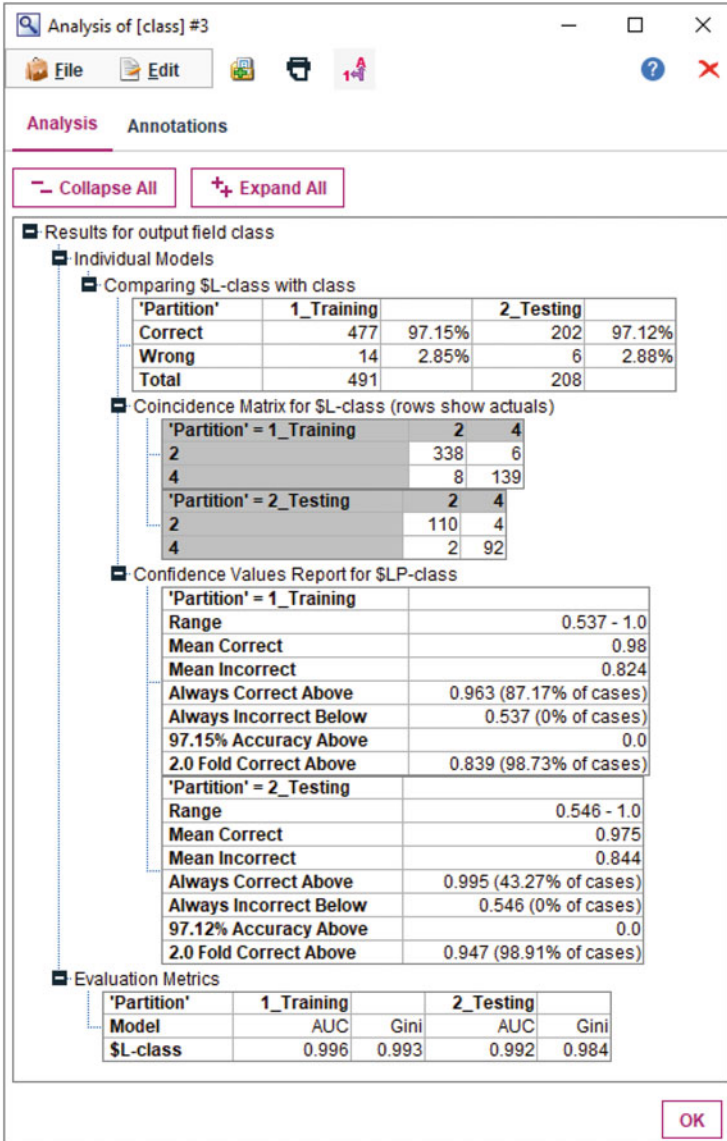


Fig. 8.16 Output of the Analysis node of a classifier with two target categories, here, a logistic regression classifier on the Wisconsin breast cancer data (see Sect. 8.3.6)

- *Include baseline.* Select this option if a reference line of the naïve random model should be included in the graph. For example, for a ROC chart, this is the diagonal which indicates the model that randomly classifies the data. This option is not available for Profit and ROI charts. We recommend to always draw this line to visualize the improvement of the prediction by the model.

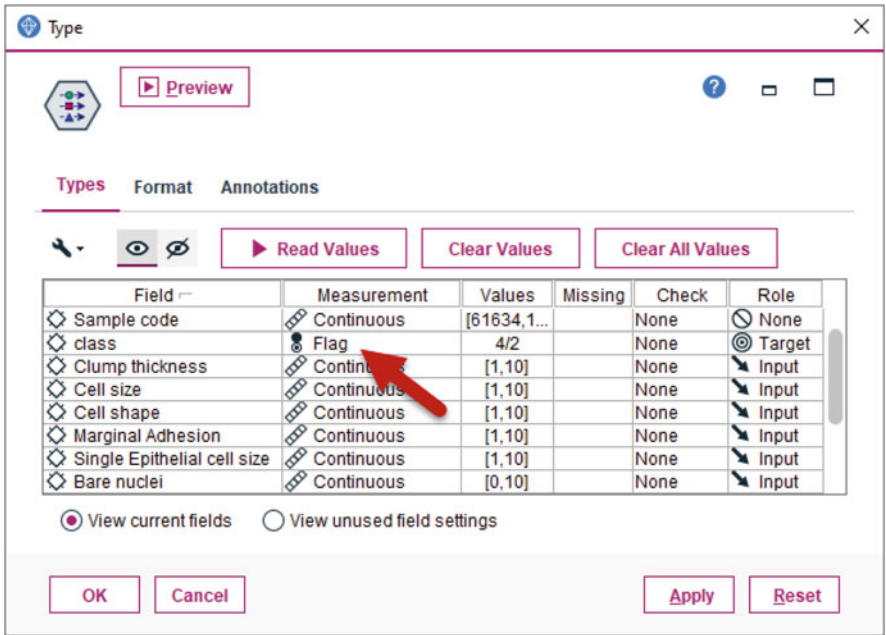


Fig. 8.17 Defining the target variable measurement as “Flag” in the Type node. This ensures correct calculation of the AUC and Gini

- *Include best line.* This option adds a line representing the optimal model, i.e., no misclassifications, to the chart. This option is not available for ROC charts.
- *Separate by partition.* For each of the partitions (training, test, validation), a separate graph is plotted if a partition field is used to split the data, e.g., with a Partition node.
- *Plot (Percentiles).* This option is available for all graphs except the ROC. Prior to the calculation of these other evolution charts, the dataset is split into quantiles, i.e., the data are sorted by the predicted score in descending order and then split into equal parts (quantiles). So, the data record with the highest scores is in the first part followed by the second highest scored data in the second part and so on. Then the required calculations are done on each quantile and the results are plotted as the desired graph. Possible options are: *Quartiles, Quintiles, Deciles, Vingtiles, Percentiles, and 1000-tiles*. Needless to say, smaller quantiles (Percentiles or 1000-tiles) ensure a smoother graph, whereas large quantiles (Quartiles or, Quintiles) generate a rougher graph.
- *Style.* With this option the style of the graph is defined. The graph can be drawn as points or as a line.
- *Profit settings.* With this option costs, revenues and weights can be specified which are associated with each record. So, instead of the default counts of hits and misses, the here defined values are summed up. The defined values can be fixed or

[SL-class]

Plot Options Appearance Output Annotations

Chart type: Receiver Operating Characteristic (ROC)

☒ Cumulative plot ☒ Include baseline ☐ Include best line

☐ Use profit criteria for all chart types

Models

Find predicted/predictor fields using:

☒ Model output field metadata

☐ Field name format (for example, "\$<x>-<target field>")

Other Score Fields

☐ Plot score fields

Target: SL-class

☒ Separate by partition

Plot: Percentiles

Style: ☒ Line ☐ Point

Costs: ☒ Fixed 5.0 ☐ Variable

Revenue: ☒ Fixed 10.0 ☐ Variable

Weight: ☒ Fixed 1.0 ☐ Variable

OK Run Cancel Apply Reset

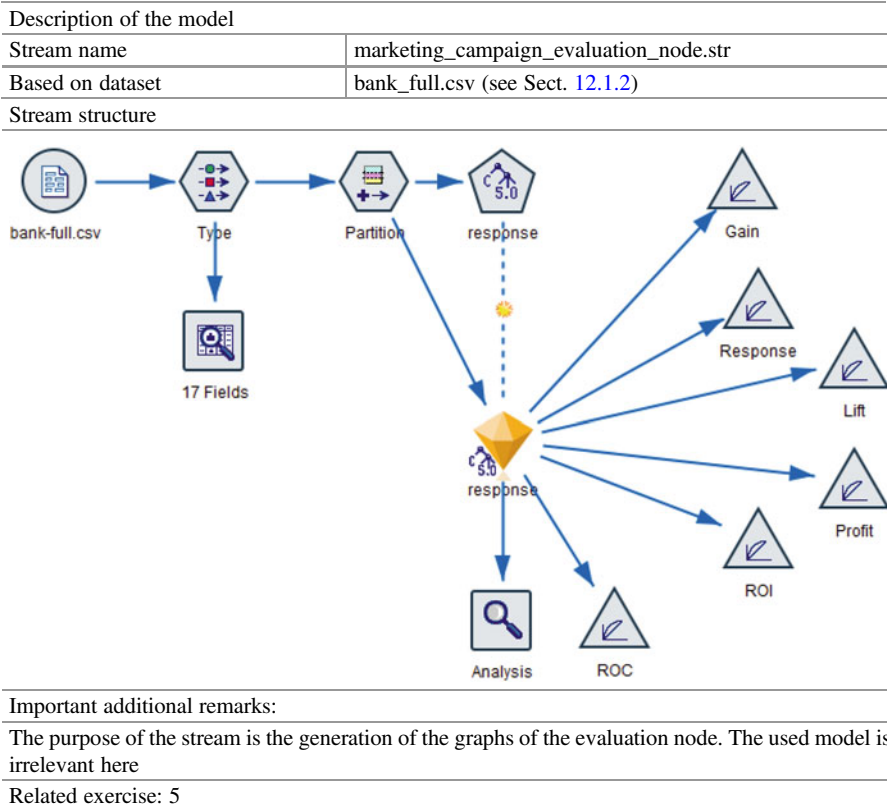
Fig. 8.18 Options of the Evolution node, here, for the visualization of the ROC curve

variable, whereby, for the latter a variable field of the dataset has to be selected. The profit settings option is mandatory for Profit and ROI chart, but can also be activated for all other graphs, except the ROC, via the 'Use profit criteria for all chart types' option at the top of the Evolution node.

In the Option tab, the user is able to define a hit, i.e., a positive case, and a scoring criterion. This ensures a flexibility to overrule the default values, which are automatically extracted from the information reported by the model. This might be beneficial if, for example, lower scores are better than higher. In this case, the hit criteria can be selected as '@TARGET <0.5' and the associated score can be defined as '1-@PREDICTED'. For further details on the Option tab, see IBM (2019c).

In the following, we describe each graph which can be plotted by the Evaluation node. The graphs are based on a direct marketing campaign case of a bank offering a

financial product. A hit (or positive event) denotes the subscription of the client to the offered product. See Sect. 12.1.2 and Moro et al. (2011) for a detailed description of the dataset. All graphs are generated by the given stream below. We omit a description of the stream here, since the goal of this section is solely the explanation of the evaluation charts.



ROC

In Fig. 8.19 the output of the Evaluation node is shown which is generated with the option settings as in Fig. 8.18. Since the “Separate by partition” option is enabled, two ROC curves are drawn, one for each partition, with the ROC curve of the training set on the left and the ROC curve of the test set on the right side. Recalling Sect. 8.2.5, the ROC curve (blue line) visualizes the FPR (or 1-Specificity) against the TPR (or Sensitivity). In addition, the reference line (diagonal), representing the model where a prediction is done randomly, is plotted in each chart. In the next Sect. 8.2.8, a detailed example of how the ROC curve is generated with the required calculations is discussed at length.

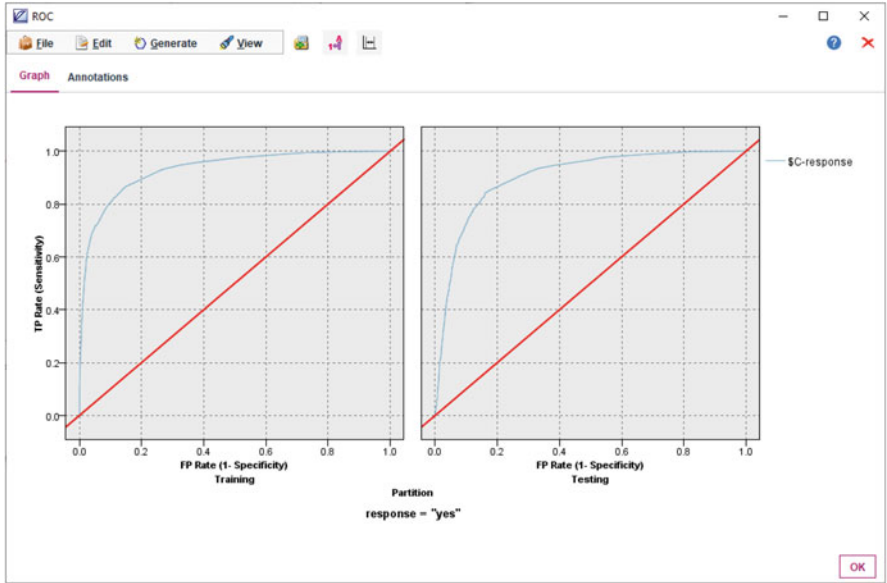


Fig. 8.19 ROC curves for the training and test set generated by the Evaluation node

We like to point to one circumstance concerning the display of the ROC curve generated by the Evaluation node (the SPSS Modeler used in this book is version 18.2). There is only one data point drawn for each value of the FPR. More precisely, if two TPRs have the same FPR, then only the point with the higher TPR is drawn. This situation occurs very often, see for example Fig. 8.12. This means that the ROC curve plotted by the Evaluation node is not exactly the (empirical) ROC curve, but an approximation which looks smoother. As a consequence, the true AUC calculated by the Analysis node (see Sect. 8.2.6) does not correspond to the ROC curve drawn. For large datasets, this inaccuracy is not visible to the human eye, but, for tiny datasets, it can attract the attention that the AUC is not exactly the area under the curve of the plotted ROC curve. See, for example, the use-case discussed in Sect. 8.2.8. However, for comparison of different models, this inexactness is good enough. Furthermore, to reduce computational time, the Evaluation node might leave out some calculations for values with $TPR = 1$, as at this point the ROC curve is a horizontal line to the point (1,1).

Gains

Gains are defined as the proportion of total hits that occur in each quantile, i.e.,

$$\text{Gain}_{\text{quantile } i} = \frac{\text{number of hits in quantile } i}{\text{total number of hits in the dataset}} \cdot 100\%.$$

Figure 8.20 shows the cumulative gain chart for the marketing campaign case together with the baseline (diagonal) and the best line (line on top). The cumulative

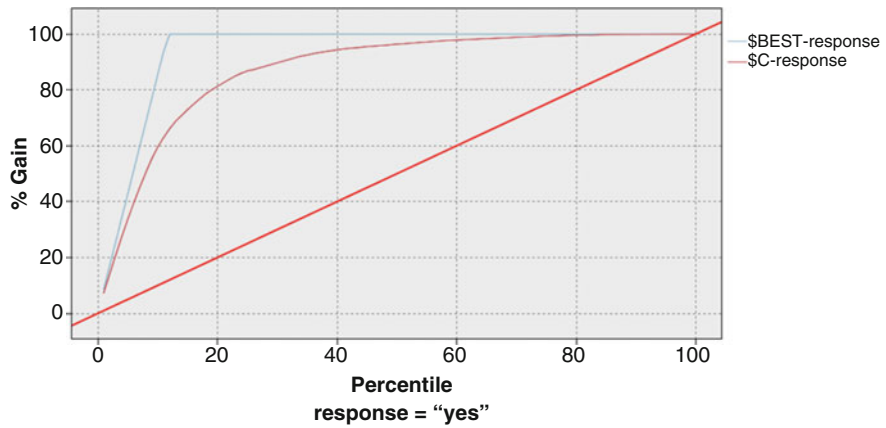


Fig. 8.20 Gain chart including the baseline and best line generated by the Evaluation node

gain chart illustrates how many responses are generated by just contacting a percentage of the “best” (highest scored) clients. For example, the top 20% of the clients are responsible for about 80% of the response (hits). On the other hand, the last 40% of the clients only generate very few subscriptions of the offered product. So, a campaign manager can consider reducing the size of the target group in order to reduce cost and, at the same time, keeping almost a similar response.

The best line, here, illustrates a gain chart of a model that would perfectly separate the responders from the nonresponders, i.e., all responders are higher scored than the nonresponders. Where the best line hits the 100% marks is the position on which the maximum number of responders is reached. So, in a perfect world, only about 16% of the clients have to be contacted in order to get the maximum response of this campaign.

Lift and Response

The Response is the percentage of records in each quantile that are hits, i.e.,

$$\text{Response}_{\text{quantile } i} = \frac{\text{number of hits in quantile } i}{\text{number of data record in quantile } i} \cdot 100\%.$$

In Fig. 8.21 the cumulative response is shown together with the baseline and the best line. The baseline, thereby, is the horizontal line and indicates the percentage of hits in the whole dataset (response rate of the total campaign), here, approximately 12%. The cumulative response chart visualizes the response rate when just concentrating on the highest $\times$ percent of the clients. For example, the best 20% of the clients have a response rate of slightly less than 50%. In other words, if the marketing campaign is just focused on the 20% clients, it has a response rate of 50%. A response graph should be monotonously decreasing until it reaches the response rate of the total dataset, i.e., the baseline. Analog to the gain chart, the best line

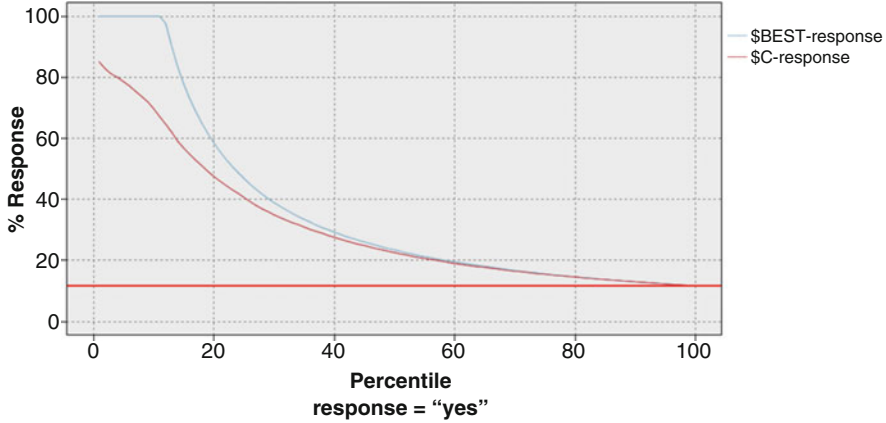


Fig. 8.21 Response chart including the baseline and best line generated by the Evaluation node

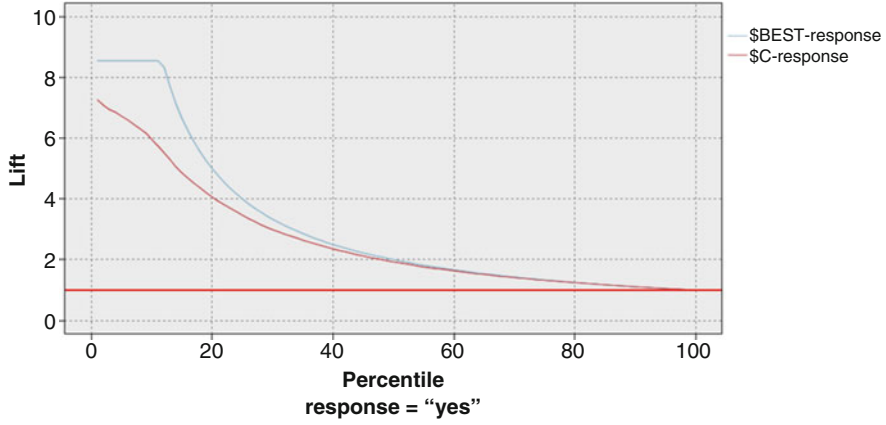


Fig. 8.22 Lift chart including the baseline and best line generated by the Evaluation node

describes a response chart for a perfect model, i.e., a model that separates the responders from the nonresponders perfectly.

The Lift is just the Response of a quantile divided through the overall percentage of hits in the dataset, i.e.,

$$\text{Lift}_{\text{quantile } i} = \frac{\text{Response}_{\text{quantile } i}}{\text{Response of the total dataset}}.$$

It measures how much a quantile performs better than the complete dataset. For a better understanding, let us have a look at Fig. 8.22, where the cumulative lift is shown with the baseline and the best line. For example, when just concentrating on the best 20% scored clients, we can get a response rate that is about four times higher

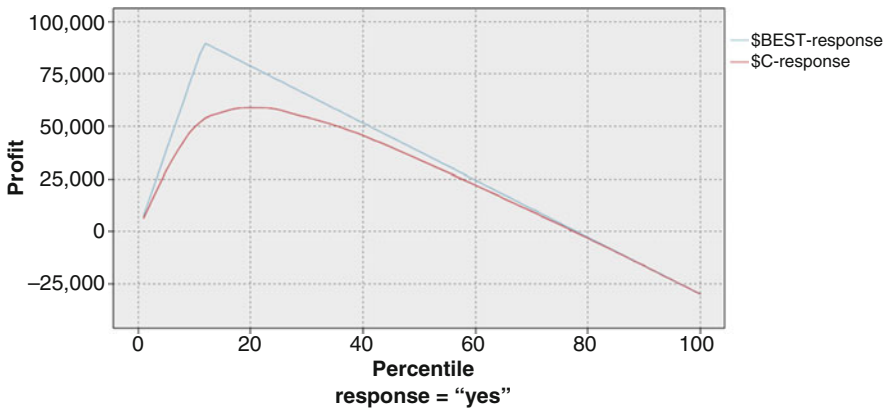


Fig. 8.23 Profit chart including best line generated by the Evaluation node

than the response rate of the entire dataset. This corresponds with the rates of the response chart. The baseline is the horizontal line at 1 and represents the entire dataset. For a good model, the lift chart should start well above 1 and then slowly descending to the baseline when going from left to right. The best line, as in the previous charts, represents the lift chart of the perfect model.

Profit and Return on Investment (ROI)

Instead of looking at the response rates, a campaign manager is often interested in the profit that is generated by the campaign. So, rather than just counting the records, a revenue and cost for each record are considered. Thus, the profit of a data record is just the revenue minus the cost for the record. Then, the profit for a quantile equals the sum of profits for all records in the quantile, i.e.,

$$\begin{aligned} \text{Profit}_{\text{quantile } i} = & \text{sum of revenue for records in quantile } i \\ & - \text{sum of costs for records in quantile } i. \end{aligned}$$

While costs apply to all records, revenues are assumed to apply only to hits. Translated to a marketing campaign, this means that calling a client or delivering an advertisement letter to a client always goes hand in hand with costs, but revenue is only generated if the client responds to the campaign and purchases a product. Costs and revenue have to be specified in the Plot tab of the Evolution node (see explanations above).

In Fig. 8.23 the cumulative profit chart is shown together with the best line. It displays the profit that is generated by the top scored clients. In this case, the top 40% of clients generate a profit of about 45,000€. This graph helps campaign managers to find the optimal size of a target group in order to maximize the profit. A profit chart usually starts near 0 and increases continuously until it reaches a peak. Then it

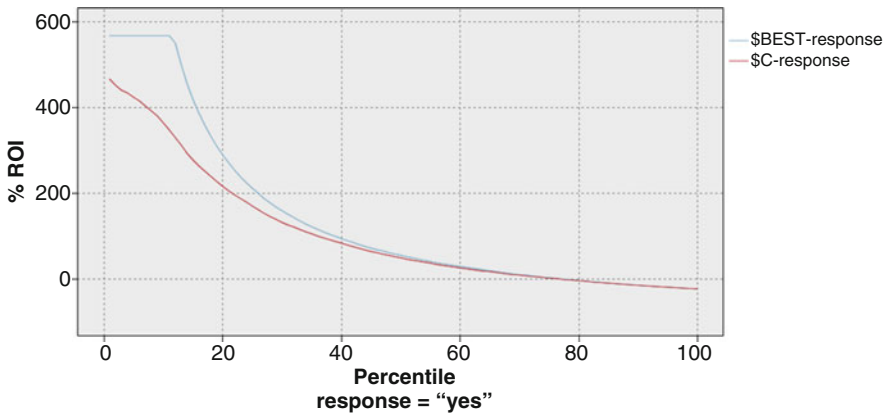


Fig. 8.24 ROI chart including best line generated by the Evaluation node

decreases steadily to the profit of the total campaign. The best line describes the profit of the perfect model, analogous to the other evaluation charts.

The ROI then compares the profit to the costs for a quantile, that is,

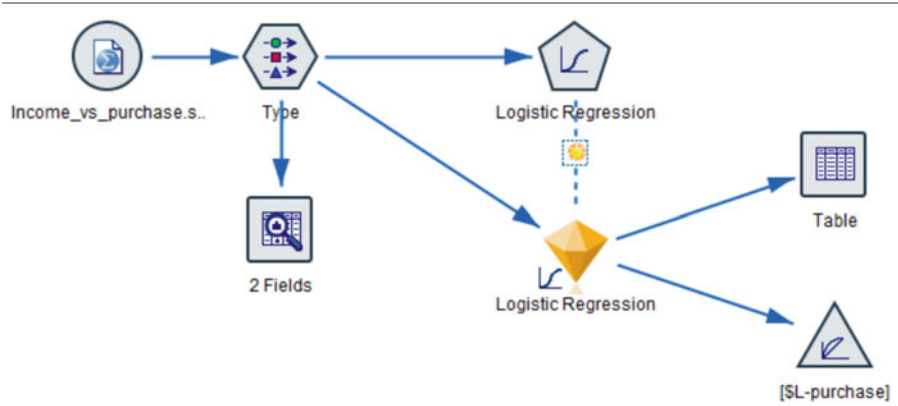
$$\text{ROI}_{\text{quantile } i} = \frac{\text{Profit}_{\text{quantile } i}}{\text{costs for the quantile } i} \cdot 100\%.$$

So, the ROI describes how much the profit exceeds the costs. In Fig. 8.24, the cumulative ROI chart is visualized together with the best line. For example, the profit of the best 20% scored clients is about 200 times higher than the costs contacting them in the marketing campaign. The ROI graph starts above 0 and steadily descends to the return of investment of the total dataset. We remark that the ROI chart is usually similarly shaped as the lift chart. The best line, finally, describes the same as in the other evaluation graphs.

8.2.8 A Detailed Example how to Create a ROC Curve

Description of the model	
Stream name	income_vs_purchase_log_reg.str
Based on dataset	income_vs_purchase.sav (see Sect. 12.1.21)
Stream structure	

(continued)



Important additional remarks:
Calculations can be found in the excel file ‘ROC_manual_calculation.xlsx’
Source of data based on Backhaus et al. (2011), p. 253; partially modified

In Sect. 8.2.5, the ROC curve was introduced with a short explanation of how it is derived from the confusion matrix on a toy example. Here, it is shown how a ROC curve is calculated manually in more detail on a simple but clear use-case. The idea and the data are taken from Backhaus et al. (2011), p. 253, with slight modifications for our purposes. The dataset describes the purchase of a certain product with respect to the monthly income of a customer. Table 8.4 shows the complete dataset, which is further visualized in Fig. 8.25.

We immediately see that all customers earning more than 3200 are buying the product. But, also one customer with monthly income of 1800 bought the product, which means that the two classes (‘purchase’ and ‘no purchase’) are not linear and separable by the income of the customer. However, for the demonstration of the ROC curve calculation, we use a linear model here and fit a binomial logistic regression model to the data. For a detailed introduction and explanation of the

Table 8.4 Dataset of the example, containing the income of a customer and an indicator if a customer purchased the product (1 = purchase and 0 = no purchase)

Purchase	Income
1	6000
1	5500
1	5200
1	5100
1	4800
1	4200
1	4000
0	3200
0	2700
0	2600
0	2500
1	1800

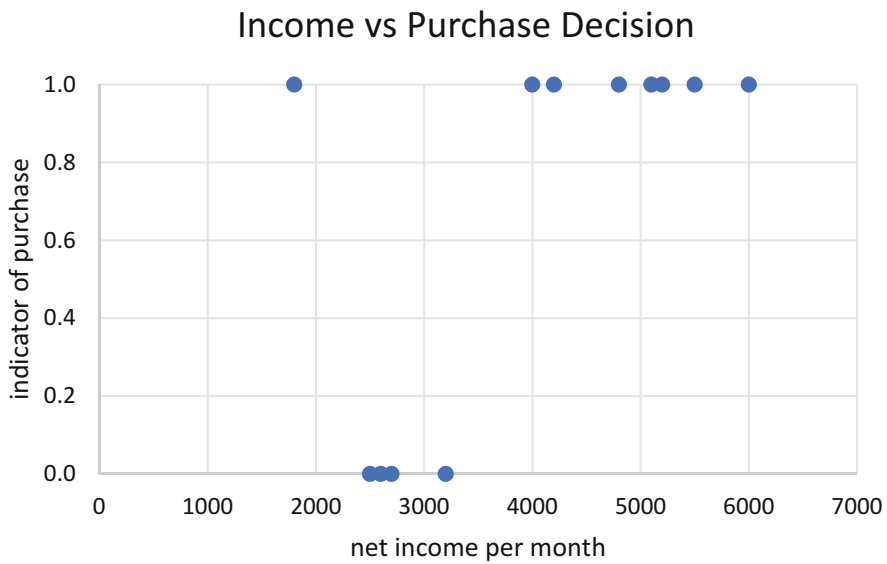


Fig. 8.25 Visualization of the dataset income vs. purchase

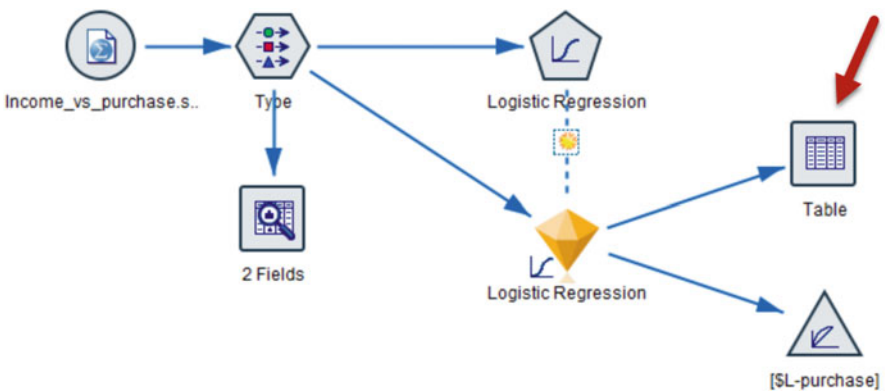


Fig. 8.26 The Table node in the stream is used to show and save the final model predictions

logistic regression, we refer the reader to the subsequent Sect. 8.3, as the specific algorithm is irrelevant in the present example and only the predictions are needed.

In the stream of this example, the logistic regression model is implemented, and its predictions can be viewed through a Table node, which is connected to the model nugget (see Fig. 8.26). Figure 8.27 shows the output of the Table node, i.e., the results of the model applied to the data.

The following columns are added to the data that contain the predictions of the logistic regression model:

\$L-purchase	Prediction of purchase yes/no resp. 1/0.
\$LP-0	Probability that there is no purchase - equals $1 - \$LP-1$.
\$LP-1	Probability that there is a purchase - equals $1 - \$LP-0$.
\$LP-purchase	Probability of the prediction \$L-purchase, also called “Associated probability”. Equals the maximum of \$LP-0 and \$LP-1.

We refer to Sect. 8.3.2 and Table 8.9 for a detailed description of the columns that are returned and added to the output table by a classification model.

Using the toolbar option ‘File → Export → Transposed Comma Delimited (*.csv)’ in the Table node, we save the data in a csv-format (see Fig. 8.28). Afterwards, we can open the file and edit the data in another data calculation tool, e.g., with Microsoft Excel, which is done hereafter. The data and all following calculations can be found in the Excel file ‘ROC_manual_calculation.xlsx’.

We like to point out that the score (the predicted probability of a success, here a purchase) in a binomial classifier completely defines the output of the model, which in particular means that all other above listed values can be derived from that prediction. For example, the column ‘\$L-purchase’ can be calculated from the purchase probability ‘\$LP-1’ as follows:

$$\$LP - 1 \geq 0.5, \text{ then } \$L - \text{purchase} = 1$$

$$\$LP - 1 < 0.5, \text{ then } \$L - \text{purchase} = 0.$$

Due to this reason and the additional fact that the predicted probabilities are more precise as the predicted class, we can only focus on the scores ‘\$LP-1’. As described in the discussion of the ROC curve in Sect. 8.2.5, we have to compare the scores with the actual class in order to calculate the ROC curve. Table 8.5 shows these two

	purchase	income	\$L-purchase	\$LP-purchase	\$LP-0	\$LP-1
1	1.000	6000...	1.000	0.988	0.012	0.988
2	1.000	5500...	1.000	0.975	0.025	0.975
3	1.000	5200...	1.000	0.961	0.039	0.961
4	1.000	5100...	1.000	0.954	0.046	0.954
5	1.000	4800...	1.000	0.929	0.071	0.929
6	1.000	4200...	1.000	0.839	0.161	0.839
7	1.000	4000...	1.000	0.793	0.207	0.793
8	0.000	3200...	1.000	0.527	0.473	0.527
9	0.000	2700...	0.000	0.660	0.660	0.340
10	0.000	2600...	0.000	0.694	0.694	0.306
11	0.000	2500...	0.000	0.726	0.726	0.274
12	1.000	1800...	0.000	0.886	0.886	0.114

Fig. 8.27 Table node with predictions of the logistic regression model

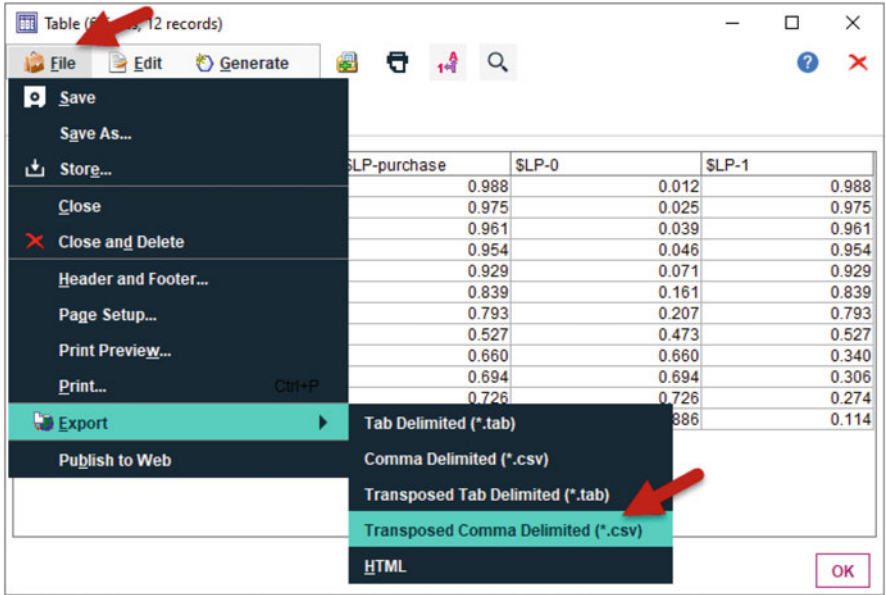


Fig. 8.28 Data export option in the Table node

Table 8.5 Output table of the logistic regression model reduced to the relevant columns, ‘\$LP-1’ and actual purchase value in descending order

	Score of a purchase calculated by the model	Actual value 1 = purchase/0 = no purchase
Row	\$LP-1	Purchase
1	0.988	1
2	0.975	1
3	0.961	1
4	0.954	1
5	0.929	1
6	0.839	1
7	0.793	1
8	0.527	0
9	0.340	0
10	0.306	0
11	0.274	0
12	0.114	1

relevant variables, ‘\$LP-1’ and actual purchase value sorted by ‘\$LP-1’ in descending order.

Before starting with the determination of the ROC curve, let us quickly recall the most important and required measures from Sect. 8.2.5 with respect to our example here. The true positives (TP) are actual purchases which are correctly classified as ‘purchase’ by the model. Furthermore, the true positive rate (TPR) is the proportion of correctly identified purchases (TP) and the total number of actual purchases (P), i.e.,

$$TPR = \frac{TP}{P}.$$

This can be interpreted as the probability that an actual purchase is classified as such. On the other hand, the true negative rate (TNR) is the proportion of correctly identified ‘no purchases’ (TN) and the number of actual ‘no purchases’ (N) in the data, i.e.,

$$TNR = \frac{TN}{N},$$

which is the probability that an actual ‘no purchase’ is correctly classified. From this, we can derive the false positive rate (FPR) via the formula

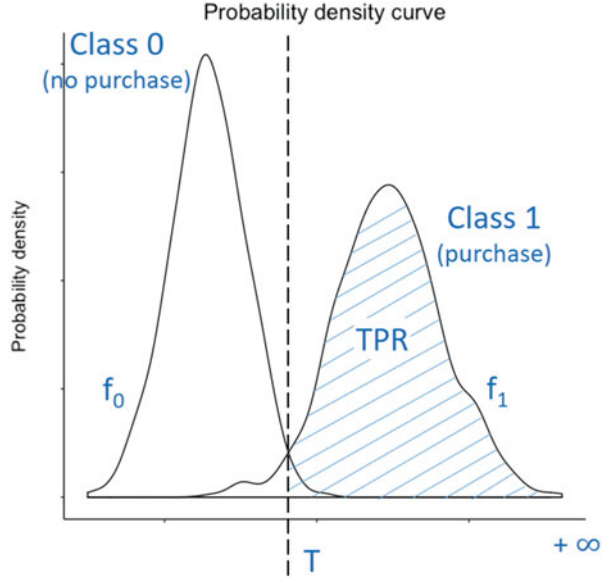
$$FPR = 1 - TNR,$$

which describes the ratio of predicted purchases that are actual ‘no purchase’ (FP) to the total number of actual ‘no purchase’ cases N . Let us recall that the ROC curve visualizes the FPR and TPR (see Sect. 8.2.5), which are thus the measures of interest from now on.

The idea of the graphs and the following explanations are based on graphs and text which can be found also on Wikipedia (2019). The goal of a classification model is to separate the actual purchases from the actual ‘no purchases’. So, a classifier predicts a probability that an instance belongs to one of the classes. Let us assume that these probabilities or scores are generated by a continuous random variable X that follows a density function $f_1(x)$ if the instance belongs to the ‘purchase’ class, and a density function $f_0(x)$ otherwise. By recalling Sect. 8.2.5, an instance is then classified as ‘purchase’ if $X > T$, and ‘no purchase’ otherwise, for a given threshold T . The density functions with the threshold are visualized in Fig. 8.29 which is based on Wikipedia (2019).

The TPR describes the probability of a correct classified ‘purchase’ data record, i.e., the probability that $X > T$ if a ‘purchase’ instance is given. But, this is nothing other than the area under the curve $f_1(x)$ right of the threshold, i.e., the integral of $f_1(x)$ from T to infinity resp. 1 since X predicts a probability,

Fig. 8.29 Probability density curves of ‘purchases’ and ‘no purchases’ and visualization of the TPR



$$TPR = \int_T^{+\infty} f_1(x) dx,$$

which is also highlighted by the shaded area in Fig. 8.29. In the same manner, the TNR, which describes the probability that $X \leq T$ if a ‘no purchase’ instance is given, equals the area under the curve $f_0(x)$ from minus infinity resp. 0 to T , i.e.,

$$TNR = \int_{-\infty}^T f_0(x) dx.$$

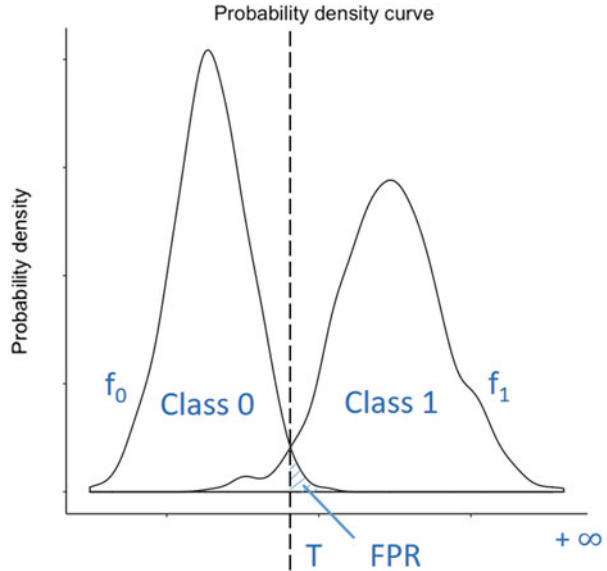
So, we deduce

$$FPR = 1 - TNR = 1 - \int_{-\infty}^T f_0(x) dx = \int_T^{+\infty} f_0(x) dx,$$

which is visualized in Fig. 8.30.

We see that the FPs are located in the overlapping area of the densities. This should not be surprising since in this overlapping interval the model is unsure of the true class, hence, leading to misclassifications of the model which can only make greater and smaller T decisions. For example, in Figs. 8.29 and 8.30, the threshold T separating the ‘purchase’ and ‘no purchase’ equals the standard threshold used by a classifier, i.e., $T = 0.5$. Let’s assume a score is just a bit larger than 0.5, e.g., 0.51,

Fig. 8.30 Visualization of the FPR in the overlapping area of the probability density curves of ‘purchases’ and ‘no purchases’



then the decision that a purchase happens is not assured, in fact, it is almost a coin flip decision. Therefore, a classifier is good if the density curves do not overlap too much and the center of the distributions is far away from each other. With more separated density curves, fewer misclassifications can occur, and the classifier provides a higher accuracy. See Fig. 8.31 for density curves with little overlapping belonging to a model with higher accuracy.

Sliding the threshold to the left and right in the graphic of the density curves, we can get a better understanding of the behavior and the dependencies between TPR and FPR as well as the quality of a classifier. Recall from Sect. 8.2.5 that for a model having strong classification power, the TPR has to be high and the FPR low. So, moving the threshold to the left would increase the TPR, but at the same time, increase the FPR as well, because a larger area of the density $f_0(x)$ is now located to the right of T . This can be illustrated in Fig. 8.32. Thus, moving the threshold to the left generates more FPs and we are in the crying wolf situation (see once again Sect. 8.2.5). On the other hand, sliding T to the right reduces the FPR as well as the TPR simultaneously. See Fig. 8.33.

After these preliminaries, we can now describe how the ROC curve is calculated using the same procedure. As the ROC curve describes the relationship of the TPR and FPR, and with it, the ability of the model to separate the two target classes, here, ‘purchase’ and ‘no purchase’, we have to determine these two measures for different thresholds T . So, starting at the right, i.e., $T = \infty$ resp. $T = 1$, we move the threshold to the left and calculate the TPR and FPR for each T . However, we only have to consider the thresholds that correspond to the predicted probabilities ‘\$LP-1\$’ since

Fig. 8.31 Visualization of the density curves with little overlap belonging to a classifier that separates the ‘purchases’ from the ‘no purchases’ very accurately

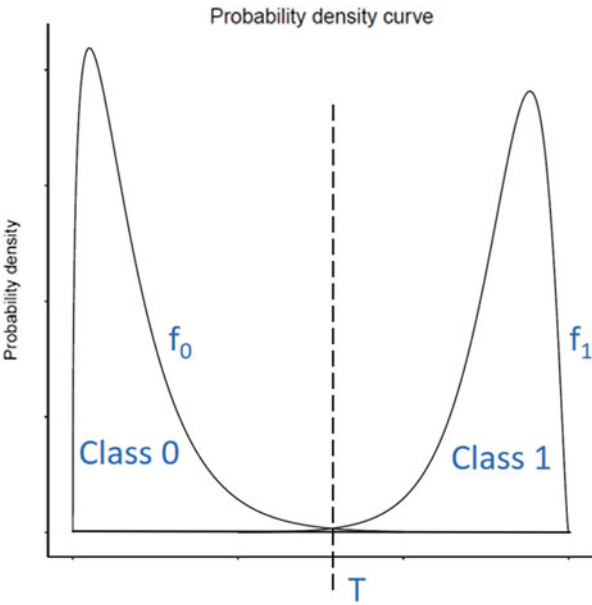
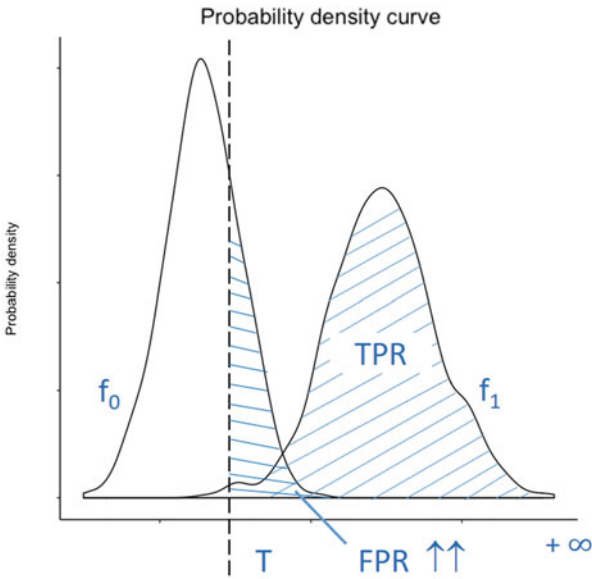


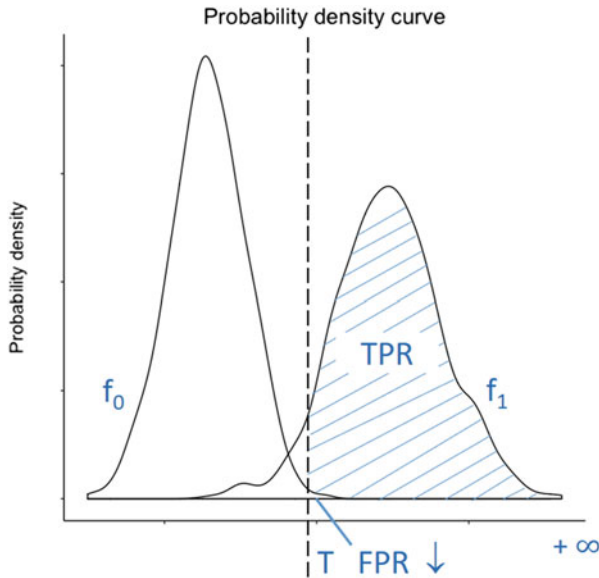
Fig. 8.32 A threshold further to the left increases the TPR as well as the FPR simultaneously



these are the only values where the classification of a data record might change, and with it, the values of the TPR and FPR.

For $T = \infty$ resp. $T = 1$, we see that all scores, and thus, both density curves, lie completely left of T , which leads to a TPR and FPR of 0. As mentioned above, we

Fig. 8.33 A threshold further to the right reduces the TPR as well as the FPR simultaneously



now start moving T to the left, stopping at every predicted probability ‘\$LP-1’ for new calculations of TPR and FPR. We recommend to always start at the right and moving from the highest to the lowest score of a purchase. For this purpose, we sort the rows of the output table of the model by the scores ‘\$LP-1’ in descending order (see Table 8.5). Now, we can perform our calculations successively from the top to the bottom of the table.

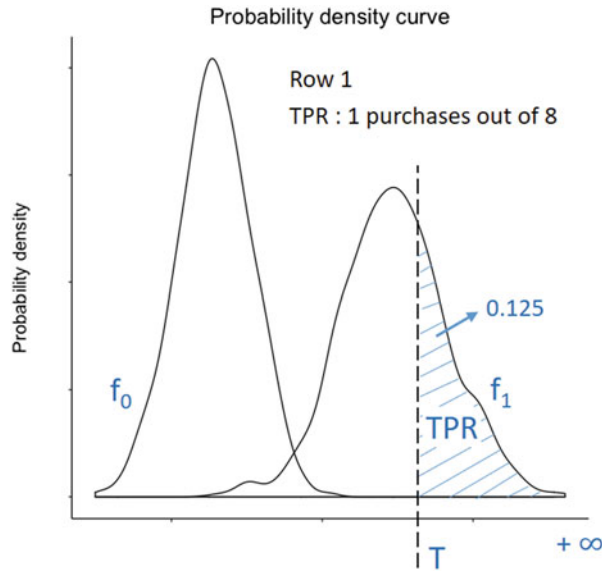
So, we start our calculations at the first row of Table 8.5 with a score of 0.988. We find the following:

Threshold	Conclusion
$T = 0.988$	Each row with a score above or equal to 0.988 will be classified as ‘purchase’, which is only one data record (the one in row 1) in this case. All other rows are assigned to the class ‘no purchase’. Since row 1 is an actual purchase, we conclude that $TP = 1$ and $FP = 0$. There are a total of eight actual purchases in the dataset, and thus, $TPR = 1/8 = 0.125$. Furthermore, $FPR = 0$. In Fig. 8.34, these calculations are visualized.

Moving our threshold line further to the left, we find in row 2 of Table 8.5:

Threshold	Conclusion
$T = 0.975$	Each row with a score above or equal to 0.975 will be classified as ‘purchase’, which are two (row 1 and row 2) in this case. Since both of these rows are actual purchases, we get that $TP = 2$ and $FP = 0$. With the same calculations as above, we get $TPR = 2/8 = 0.250$ and $FPR = 0$.

Fig. 8.34 Probability density curve and threshold for row 1 of Table 8.5



Following the same procedure as for the first two rows, we can calculate the TPR and FPR for each threshold of the first seven rows. Thus, in every step, the TPR increases by 0.125 up to $TPR = 7/8 = 0.875$ for row 7, where $T = 0.793$. Meanwhile, the FPR stays at 0 for all of these rows.

So next, let us have a look at row 8. We get:

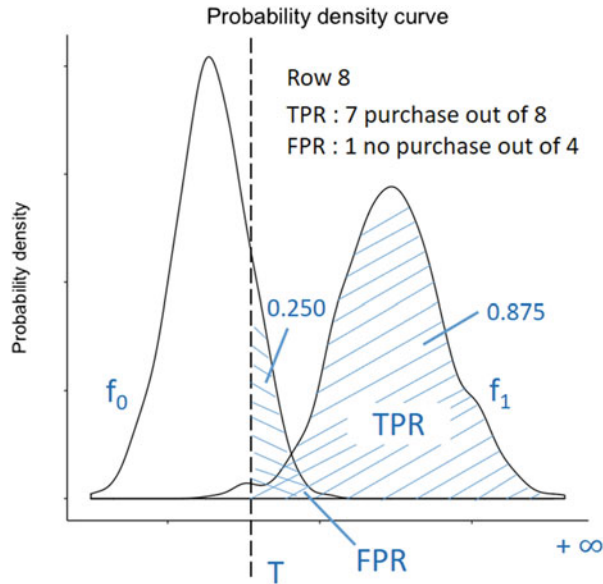
Threshold	Conclusion
$T = 0.527$	Each row with a score above or equal to 0.527 will be classified as ‘purchase’, i.e., the first eight rows. The first seven rows are actual purchases, whereas row 8 belongs to a ‘no purchase’ data record. Se, we have our first false positive classification, leading to $TP = 7$ and $FP = 1$. Still, $TPR = 7/8 = 0.875$, but, for the first time, the FPR is positive. Since there are a total of four ‘no purchases’ cases in the data, we get $FPR = 1/4 = 0.250$. See Fig. 8.35 for a visualization of this case.

We continue with this procedure for every following row in Table 8.5 until we arrive in the last row 12, and we determine the final values for the ROC curve.

Threshold	Conclusion
$T = 0.114$	Each row with a score above or equal to 0.114 will be classified as ‘purchase’, which are all records in the dataset. So, we obtain $TP = 8$ and $FP = 4$, and we get that $TPR = 8/8 = 1$ and $FPR = 4/4 = 1$.

All the values of the above calculations are listed in Table 8.6. Besides the output of the classifier, the TP, FP TPR, and FPR are listed for each row, which means that the threshold in this case is the score ‘\$LP-1’ of that particular row. Furthermore, the

Fig. 8.35 Probability density curve and threshold for row 8 of Table 8.5, where we find $TPR = 0.875$ and $FPR = 0.250$



column ‘AUC’ contains the increment of the area under the ROC curve from the previous row to the particular one.

Finished with our manual calculations, we can finally visualize the ROC curve, which is displayed in Fig. 8.36, by plotting the values of the columns ‘False Positive Rate’ against ‘True Positive Rate’. When comparing the shape of the ROC curve with the process of moving the threshold through Table 8.6 from top to bottom, we observe that if a hit (‘purchase’) is seen, the ROC curve goes up and if a miss (‘no purchase’) is seen the ROC curve goes one step to the right. So, the (empirical) ROC curve has the form of a step function in this simple case. The reason is that each predicted score is unique in the dataset. If, otherwise, ties occur, that means, numerous sample in the dataset have the same predicted score, multiple steps are done at once and in this region, the ROC curve is a line with positive and finite slope.

The calculation of the AUC (area under the curve) is pretty simple since the ROC curve is a step function in our example. The AUC is just the sum of all rectangles generated by two successive rows. Thus, the width of the rectangle is the difference of the FPR with the FPR of the previous row, whereas the height is calculated with the formula

$$m = \frac{TPR_{previous} + TPR}{2}$$

In the Microsoft Excel file ‘ROC_manual_calculation.xlsx’, we can now implement the formula as shown in Fig. 8.37. By summing up all increments, we get an AUC of 0.875, which is the exact value calculated by Analysis node.

Table 8.6 The output of the classifier extended by the values of the above calculations. In each row, the TP, FP TPR, FPR, and AUC increments are listed for the threshold being the score '\$LP-1' of the particular row

Row	Score of a purchase calculated by the model	Actual value 1 = purchase/ 0 = no purchase	Number of true positives	number of false positives	True positive rate	False positive rate	Area under the curve
	\$LP-1	Purchase	TP	FP	TPR = TP/ P	FPR = FP/ N	AUC
1	0.988	1	1	0	0.1250	0.0000	0.0000
2	0.975	1	2	0	0.2500	0.0000	0.0000
3	0.961	1	3	0	0.3750	0.0000	0.0000
4	0.954	1	4	0	0.5000	0.0000	0.0000
5	0.929	1	5	0	0.6250	0.0000	0.0000
6	0.839	1	6	0	0.7500	0.0000	0.0000
7	0.793	1	7	0	0.8750	0.0000	0.0000
8	0.527	0	7	1	0.8750	0.2500	0.2188
9	0.340	0	7	2	0.8750	0.5000	0.2188
10	0.306	0	7	3	0.8750	0.7500	0.2188
11	0.274	0	7	4	0.8750	1.0000	0.2188
12	0.114	1	8	4	1.0000	1.0000	0.0000

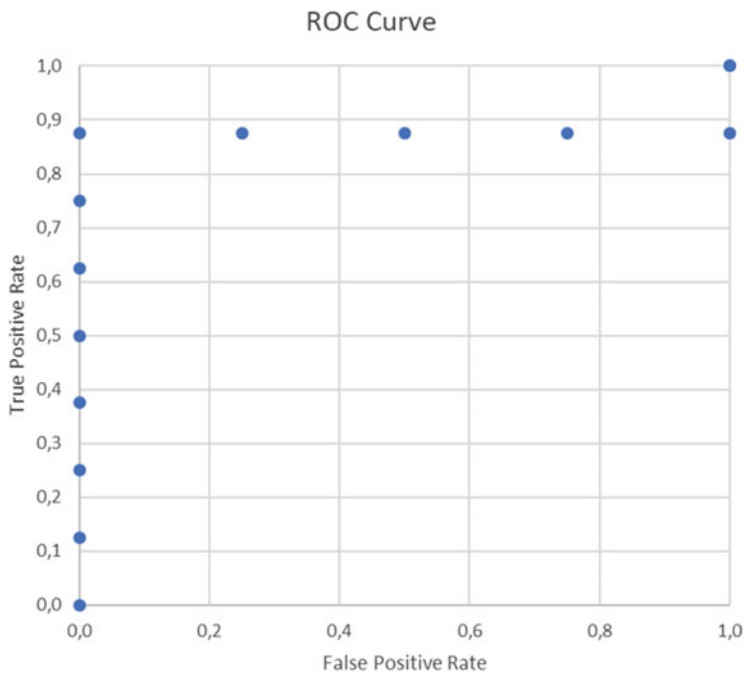


Fig. 8.36 Graph of the manually calculated ROC curve, which displays the TPR and FPR of Table 8.6

	A	B	C	D	E	F	G	H	I
1			1 = purchase 0 = no purchase	cumulative					
2	row	score \$LP-1	purchase = actual class	TP	FP	TPR = TP / P	FPR = FP / N	AUC	
12	9	0,340	0	7	2	0,8750	0,5000	0,2188	
13	10	0,306	0	7	3	0,8750	0,7500	0,2188	
14	11	0,274	0	7	4	0,8750	1,0000	=(F13+F14)/2*(G14-G13)	
15	12	0,114	1	8	4	1,0000	1,0000	0,0000	
16							Sum:	0,8750	

Fig. 8.37 AUC calculation of the example in Microsoft Excel

Let us pause here for a moment and have a look at the ROC curve that is generated by the SPSS Modeler for this example. The ROC curve plotted by the Evaluation node can be seen in Fig. 8.38. We notice that the curve looks different at the right side than our manually calculated graph in Fig. 8.36. The data point of row 11 in Table 8.6 i.e., (1, 0.875), seems missing. Also, the area under the curve of this ROC curve is obviously larger than 0.875, which does not match the calculated AUC by the Analysis node. This is an example of the inaccuracy of the ROC curve generated by the Evaluation node, which was mentioned in Sect. 8.2.7.

We like to mention that the calculations of the ROC curve and the AUC are simple in our example, but can be a bit more complicated in practice, for example,

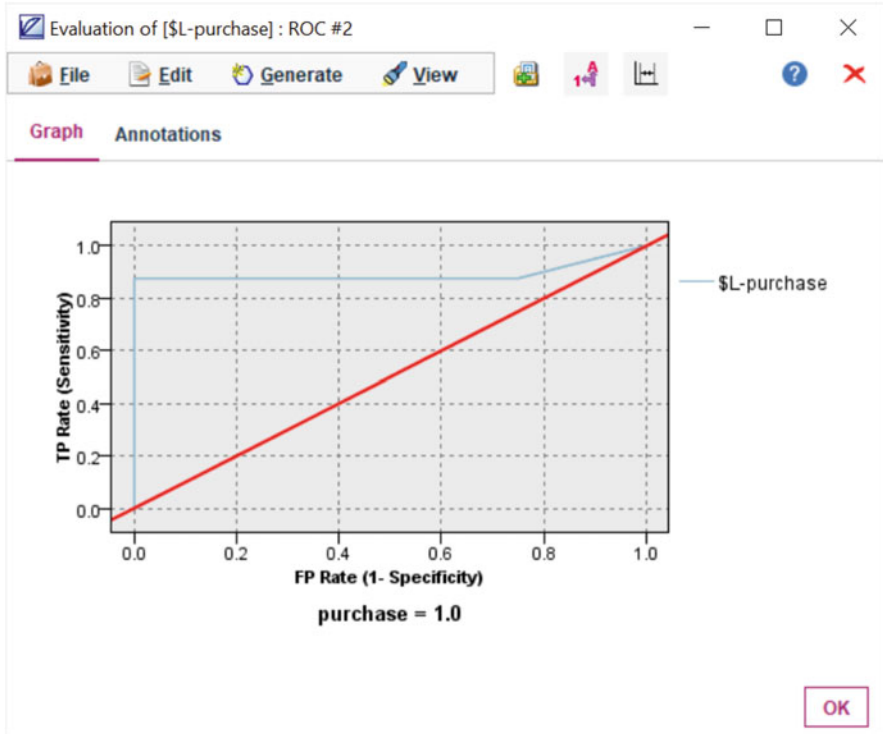


Fig. 8.38 ROC curve generated by the Evaluation node of the SPSS Modeler

when ties occur and multiple data records have the same scores. In this case, the algorithm described here has to be slightly modified.

For the interested reader, we refer to Tom Fawcett (2003) for a description of modifications of the algorithm that calculates the ROC curve.

8.2.9 Exercises

Exercise 1: Building a Classifier and Prediction for New Data

1. Explain in your own words the steps for building a classifier.
2. Consider a classifier for the assignment of a “good” or “bad” credit rating to bank customers. The classifier has the following set of rules:

Set of Rules	
If <i>Income</i> = „low“ then <i>Credit_rating</i> = „bad“.	
If <i>Income</i> = „high“ then <i>Credit_rating</i> = „good“.	
If <i>Income</i> = „medium“ and (<i>Number_of_credit_card</i> = „<5“ or <i>Age</i> = „> 28“) then <i>Credit_rating</i> = „good“.	
If <i>Income</i> = „medium“ and <i>Number_of_credit_card</i> = „>4“ and <i>Age</i> = „<= 28“ then <i>Credit_rating</i> = „bad“.	

Use this classifier to predict the credit rating of the following customers:

Name	Age	Income	Number of credit cards
John	21	Low	>4
Frank	34	Medium	>4
Sandy	31	Medium	>4
Andrea	28	Medium	>4
Lily	32	Low	>4
Ted	24	Medium	>4
Penny	34	Medium	<5
Howard	52	Medium	<5
Alfred	28	High	>4
Bruce	56	High	<5
Claire	22	High	>4
Walter	30	Low	>4
Werner	29	Medium	<5

Exercise 2: Calculation of Goodness of Fit Measures

Recall the example of credit card rating from Exercise 1. Assume now that the classifier uses a scoring function that calculates the probability that a customer has a “good” credit rating. The probabilities of the customers and the true credit ratings are as follows:

Name	Prob. for good credit rating	True credit rating
John	0.18	Bad
Frank	0.56	Bad
Sandy	0.56	Bad
Andrea	0.20	Bad
Lily	0.18	Bad
Ted	0.20	Bad
Penny	0.86	Good
Howard	0.86	Good
Alfred	0.82	Good
Bruce	0.97	Good
Claire	0.82	Good
Walter	0.18	Good
Werner	0.86	Good

1. Predict the credit rating in cases where the classifier suggests a “good” rating, if the probability is greater than 0.5, and a “bad” rating otherwise. What are the number of TP, FP, TN, and FN and the accuracy in this case?
2. Draw the ROC curve and calculate the AUC from the probabilities of a “good” credit rating and a true credit rating.

Exercise 3: Pitfalls in Classification

When building a classification model, there are many factors you have to keep in mind, which can negatively influence the ability of the prediction model and lead to worse models. These include:

- *Imbalanced data*: The Target variable is very unevenly distributed in your training data. For example, in a binary target variable there is only 5% of 1 and 95% of 0. This can happen in fraud data, for example, since fraud is rare compared to normal use.
- *Outliers*: These are data points or variable values that are located far away from all the others. These can result from measurement errors, for example.
- *Missing values*: Data may not always be complete, and some data records contain missing values. That means, not every attribute of a data record has a proper value, i.e., the data of this variable is missing. This incompleteness can occur if, for example, the data are unavailable like customer information for sales transaction data, or the variable weren’t considered to be important in the past.
- *A huge number of predictor variables*: The number of input variables is pretty high compared to the number of observations. One example is gene data, which comprise a huge number of measured values, but samples and tissues of a particular disease are often rare, so the number of observations is very few.

Give reasons why these effects can worsen your model and eliminate a good prediction. Explain the term “overfitting” in your own words.

Exercise 4: Confusion Matrix and Type of Error

We look at customers entering a supermarket. They are interested in a certain product and buy it in the shop or not. Based on historical data, we build a statistical model to predict the purchase. Purchasing the product is being defined here as the positive case.

		Predicted Values	
		positive	negative
Actual Values	positive	A	B
	negative	C	D

1. Describe the different table cells in form of: True Positives, False Positives, True Negative, and False Negative.
2. Which cell represents the Type I resp. the Type II error?
3. Recall from Sect. 8.2.5 that the measure *recall* is calculated as

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}},$$

and describes the proportion of correctly identified positive cases to all actual positive cases, e.g., the proportion of number of customers who actually bought a product and(!) also predicted as such divided by all customers who actually bought it.

A similar measure is the *precision* which is calculated as

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}.$$

and it describes the proportion of correctly identified positive cases to all positively predicted cases, e.g., the number of customers who actually bought a product and(!) also predicted as such divided by all customers who are predicted as buyers. Precision is also referred to as positive predictive value (PPV).

Mark the relevant cells in the confusion matrix which are used for the calculation of recall and precision. Give two examples, one of a prediction having a high recall and low precision, and one with a low recall and high precision.

Exercise 5: Interpretation of Evaluation Charts

Recall the example used in Sect. 8.2.7, which comprises information of a direct marketing campaign for a financial product of a bank (see Moro et al. (2011)). Assume you are a marketing manager looking at the evaluation graphs (Figs. 8.19–8.24) in order to optimize the campaign. Hint: The figures show the evaluation graphs for the specified target group and expected response of the campaign.

1. You are concerned that the size of the target group is too large, but your colleague wants to stick with the planned target group. Which of the evaluation graphs proves your point and how do you convince your colleague to reduce the target group size?
2. Mark the area in the profit chart in which you would pick the size of the target group. Give a reason for your choice.
3. Despite the profitability of the campaign, at least 90% of the total hits (response) have to be in your final target group. How large do the final target group has to be? Which graph do you use to answer the question? What response rate can you expect for the campaign with the new chosen target group size?

Fig. 8.39 Predicted credit ratings

Name	Credit rating
John	Bad
Frank	Good
Sandy	Good
Andrea	Bad
Lily	Bad
Ted	Bad
Penny	Good
Howard	Good
Alfred	Good
Bruce	Good
Claire	Good
Walter	Bad
Werner	Good

Table 8.7 Values of TP, FP, TN, FN for the credit card

		Predicted credit card rating	
		Yes	No
True credit card rating	Yes	TP = 6	FN = 2
	No	FP = 1	TN = 4

8.2.10 Solutions

Exercise 1: Building a Classifier and Prediction for New Data

Theory discussed in section	Sect. 8.2.1
-----------------------------	-------------

1. This has been explained in Sect. 8.2.1, and we refer to this chapter for the answer to this first question.
2. The predicted credit ratings are listed in Fig. 8.39.

Exercise 2: Calculation of Goodness of Fit Measures

Theory discussed in section	Sect. 8.2.4
	Sect. 8.2.5

1. The predicted credit ratings are the same as in Exercise 1 and can be viewed in Fig. 8.39. Table 8.7 shows the confusion matrix with TP, FP, FN, TN
With these values, we obtain the accuracy

Thresholds	FPR	TPR
0	1	1
0.18	1	1
0.20	0.666	0.857
0.56	0.333	0.857
0.82	0	0.857
0.86	0	0.571
0.97	0	0.143
1	0	0

Fig. 8.40 FPR and TPR for the relevant threshold of the ROC curve

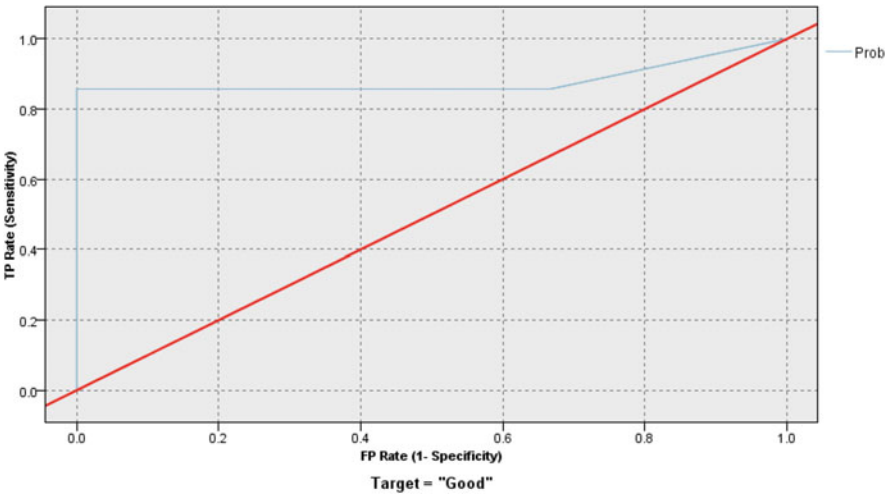
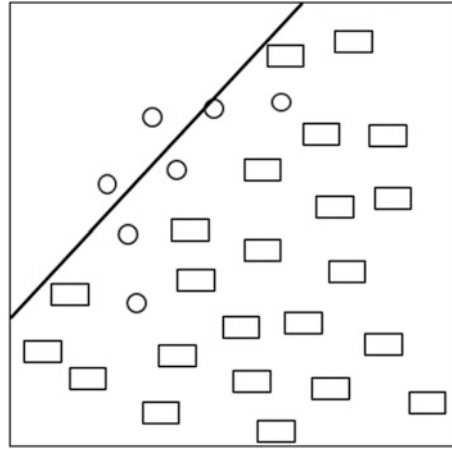


Fig. 8.41 ROC curve of the credit rating prediction

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{6 + 4}{13} = 0.231.$$

2. The values of the TPR and FPR for the relevant thresholds are listed in Fig. 8.40, and they are calculated as described in Sect. 8.2.8. The ROC curve then looks like Fig. 8.41

Fig. 8.42 Visualization of the imbalanced data problem. The minority class is highly misclassified



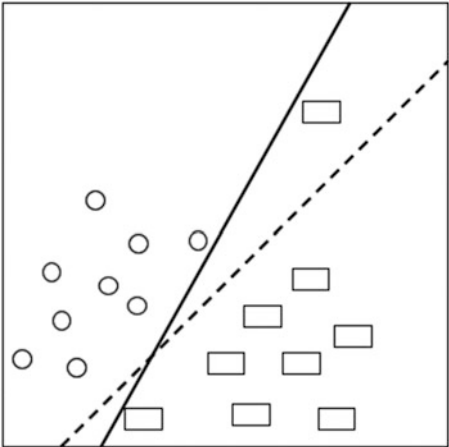
Exercise 3: Pitfalls in Classification

- *Imbalanced data:* Classifier algorithms can prefer the majority class and optimize the model to predict this class very accurately. As a consequence, the minority class is then poorly classified. This leaning towards the larger target class is often the optimal choice for highest prediction accuracy, when dealing with highly imbalanced data.

In Fig. 8.42, the imbalanced data problem is visualized. The dots are underrepresented compared with the rectangles, and thus the decision boundary is matched to the latter class of data points. This leads to high misclassification in the dots' class, and hence, 5 out of 7 dots are misclassified, as they lie on the wrong side of the boundary. In Chap. 10, this problem of imbalanced data is discussed extensively and a variety of methods for dealing with this problem are presented. In addition to Chap. 10, we refer the interested reader to Haibo He and Garcia (2009) for details on this issue.

- *Outliers:* An outlier in the data can vastly change the location of the decision boundary and, thus, highly influence the classification quality. In Fig. 8.43, the calculated decision boundary is heavily shifted, in order to take into account the outlier at the top. The dotted decision boundary might be a better choice for separation of the classes.
- *Missing values:* There are two pitfalls when it comes to missing values. First, a data record with missing values leaks information. For some models, these data are not usable for training, or they can lead to incorrect models, due to misinterpretation of the distribution or importance of the variables with missing values. Moreover, some models are unable to predict the target for incomplete data. There are several methods to handle missing values and assign a proper value to the missing field. See Chap. 11 and Han et al. (2012) for a list of concepts to deal with missing values.
- *A huge number of predictor variables:* If the ratio of input variables to the number of observations is extremely high, classifiers tend to be overfitting. The more

Fig. 8.43 Visualization of the outlier problem. The decision boundary is not chosen optimally, due to the outlier at the top



input variables exist, the more possibilities for splitting the data. So, in this case, we can definitely find a way to distinguish between the different samples through at least one variable. One possible technique for dealing with this problem is dimension reduction, via PCA or factor analysis (see Chap. 6). For further information, we refer to James et al. (2013).

Overfitting is a phenomenon that occurs when the model is too well-fitted to the training data, such that it is very accurate on the training data, but inaccurate when predicting unseen data. See Sect. 8.2.4, Fig. 8.9, and Sect. 5.1.2.

Exercise 4: Confusion Matrix and Type of Error

Theory discussed in section	Sect. 8.2.5
-----------------------------	-------------

1. Figure 8.44 shows the confusion matrix with the correct description of the cells.
2. The types of error are also assigned to the confusion matrix cells shown in Fig. 8.44.
3. Figs. 8.45 and 8.46 visualize the cells relevant to the calculation of recall and precision. As can be seen, the true negatives are not taken into account in both cases.

For the example of a high recall and low precision, let us stay at the purchase use-case. Assume, that we have a dataset of 100 customers with 10 positive (purchase) and 90 negative (no purchase) cases. Now, a model that identifies all positive cases is the naïve model that always predicts ‘purchase’, i.e., $TP = 10$. However, it also misclassifies the ‘no purchase’ cases as purchases, hence, $FP = 90$ and $FN = 0$. So, we get a high recall and low precision,

		Predicted Values	
		positive	negative
Actual Values	positive	True Positives	False Negatives Type II Error
	negative	False Positives Type I Error	True Negatives

Fig. 8.44 Confusion matrix with description of the cells including the types of errors

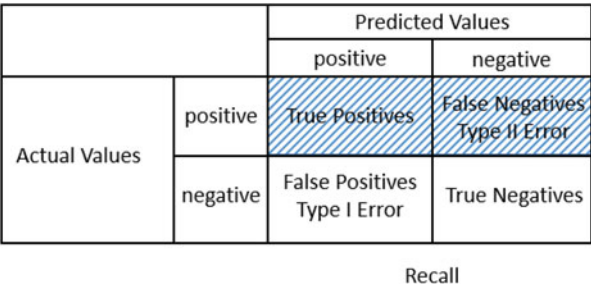


Fig. 8.45 Visualization of recall

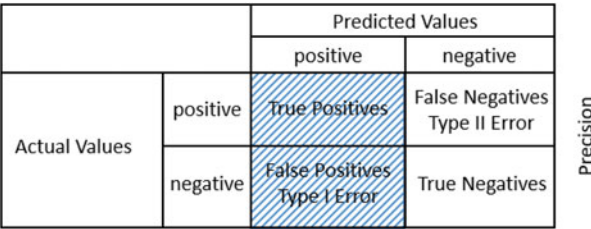


Fig. 8.46 Visualization of precision

$$\text{Recall} = \frac{10}{10 + 0} = 1 \quad \text{and} \quad \text{Precision} = \frac{10}{10 + 90} = 0.1.$$

Let us now assume, the model is trained to be very “picky”, that is, it only predicts ‘purchase’ if it is really sure. So, for example, such a model predicts that 2 of the 10 positive cases correctly, i.e., $TP = 2$, and all other data points are

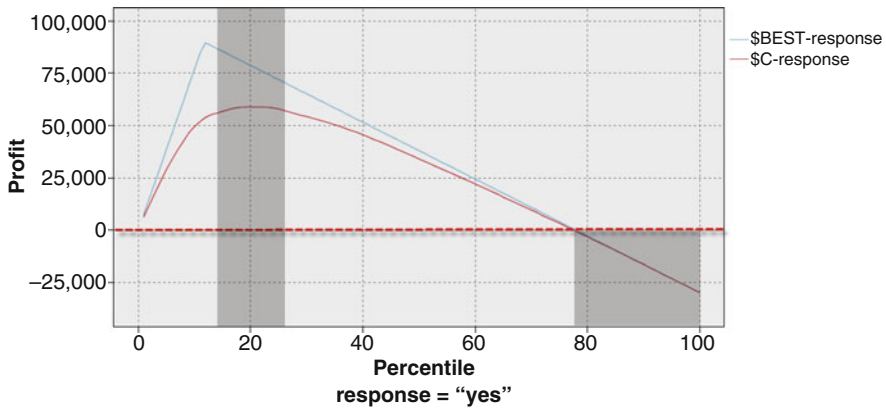


Fig. 8.47 Profit chart with highlighted negative profit area and optimal target group size for maximal profit for the marketing campaign

classified as ‘no purchase’, which results in $FP = 0$ and $FN = 8$. So, we get a low recall and high precision,

$$\text{Recall} = \frac{2}{2+8} = 0.2 \quad \text{and} \quad \text{Precision} = \frac{2}{2+0} = 1.$$

Exercise 5: Interpretation of Evaluation Charts

1. We choose the cumulative profit chart and see that the campaign with the target group as planned is expected to generate a negative profit. As can be seen in Fig. 8.47, the profit line is under 0 at the far right. This means that the campaign with the planned target group would generate a negative profit, and a reduction of the target group to the best scored clients in the area where the profit line is above 0 would be better.
2. In the profit chart Fig. 8.47 the area between the top 15% and 25% scored clients is marked since in this interval the campaign generates the most profit. So, reducing the target group of top scored clients to a size in this area would be the optimal choice.
3. To gain at least 90% of the total response, we have to select approximately the top 32% scored clients for the campaign. This can be determined by the gain chart, which is shown in Fig. 8.48. There, the dotted lines mark the position where at least 90% of the response is captured by the target group.
If the final campaign is just targeting the top 32% clients, we can expect a response rate of around 34%, see the response chart in Fig. 8.49.

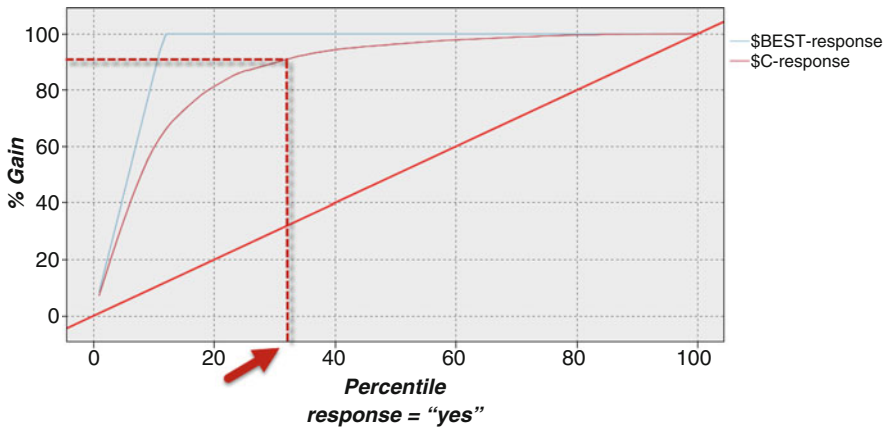


Fig. 8.48 Gain chart of the marketing campaign. The dotted lines mark the position where at least 90% of the response is captured by the target group

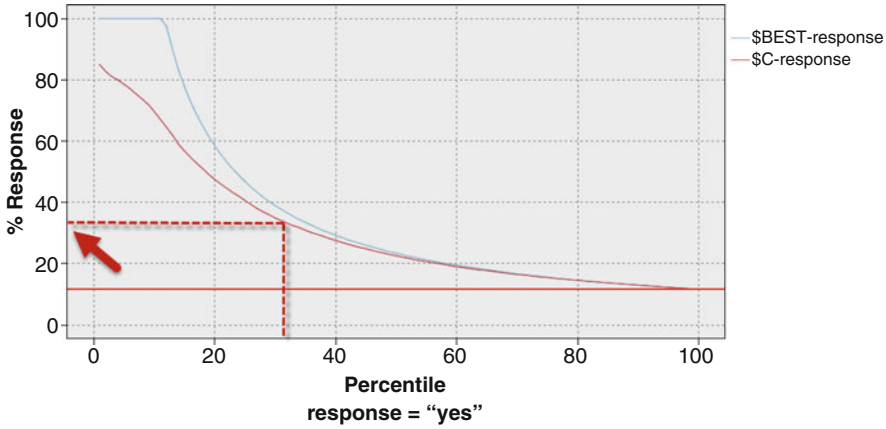


Fig. 8.49 Response chart of the marketing campaign with the expected response rate of the top 32% clients of about 34%

8.3 Logistic Regression

Logistic regression (LR) is one of the most famous classification models and is used for many problems in a variety of fields. It is so incisive and of such relevance and importance, especially in the financial sector, that the main contributors to the theory received a Nobel prize in economy in 2000 for their work. LR is pretty similar to Linear Regression described in Chap. 5. The main difference is the scale of the target value. Linear regression assumes a numeric/continuous target variable and tries to estimate the functional relationship between the predictors and the target variables,